

Make Each Token Count: Towards Improving Long-Context Performance with KV Cache Eviction

Ngoc Bui^Y, Hieu Trung Nguyen^C, Arman Cohan^Y, Rex Ying^Y

^YDepartment of Computer Science, Yale University, ^CThe Chinese University of Hong Kong

🔗 Source Code: <https://github.com/ngocbh/trimkv>

Abstract

The key-value (KV) cache is a major bottleneck in long-context inference, where memory and computation grow with sequence length. Existing KV eviction methods reduce this cost but typically degrade performance relative to full-cache inference. Our key insight is that full-cache attention is not always optimal: in long contexts, irrelevant tokens can dilute attention away from useful evidence, so selective, learnable eviction can improve generation rather than merely approximate the full cache. We introduce a global retention-based KV eviction method that learns each token’s future utility under a unified memory budget. Lightweight retention gates assign utility scores to cached KV entries, and a shared final scoring projection calibrates these scores across all layers and heads. This enables a single global eviction policy in which tokens from different layers, heads, and modalities compete directly for cache capacity. We further provide theoretical analysis showing that preferentially retaining useful tokens reduces attention dilution, and we justify geometric retention as a query-agnostic proxy for future utility. Across diverse long-context language and vision-language reasoning, and multi-turn dialogue benchmarks, our method substantially reduces KV memory while matching or surpassing full-cache inference. These results suggest that learned, globally calibrated KV eviction is not only a compression technique, but also a mechanism for improving long-context reasoning.

1. Introduction

The key-value (KV) cache is central to efficient autoregressive decoding in transformer-based language and vision-language models (LLMs and VLMs). By storing past keys and values, the model avoids recomputing representations for previous tokens during generation. However, this cache grows linearly with sequence length, and the attention computation over cached tokens grows with the amount of retained context. This becomes a major bottleneck in long-context and multimodal settings, where prompts may contain tens of thousands of text tokens or hundreds to thousands of visual tokens from images and videos [Huang et al., 2025a; Sapkota et al., 2025; Tu et al., 2024]. As context windows continue to expand, KV cache management has become one of the main challenges for practical long-context inference.

A common solution is KV eviction: once the cache exceeds a memory budget, the system removes tokens estimated to be unimportant. Existing methods typically view eviction as a compression problem, aiming to approximate full-cache inference while reducing memory and computation [Li et al., 2024b]. Many policies rely on heuristics such as recency, accumulated attention, or local attention magnitude [Xiao et al., 2023; Li et al., 2024c; Zhang et al., 2023; Cai et al., 2025]. While effective at reducing cost, these methods often degrade model quality relative to full-cache inference. This degradation is usually treated as unavoidable: removing context is assumed to trade accuracy for efficiency.

We revisit this assumption. Our key insight is that full-cache inference is not always ideal in long contexts. When many irrelevant or weakly relevant tokens remain in the cache, self-attention must normalize over all of them. As a result, useful evidence competes with a growing number of distractors,

and attention mass can be diluted away from the tokens needed for prediction [Bansal et al., 2026]. From this perspective, KV eviction is not merely an approximation to full-cache attention. If the right tokens are removed, eviction can suppress distractors, sharpen attention, and improve generation.

This perspective raises two central questions. First, *how can a model identify which cached tokens will remain useful for future decoding?* Attention-based eviction heuristics are limited because attention scores are query-dependent and often reflect short-term relevance to the current prediction, rather than persistent utility across later decoding steps, subproblems, turns, or modalities. Second, *how should a limited KV budget be allocated across layers and heads?* Different layers and heads may serve different roles: some preserve long-range information, while others mainly attend to local or short-lived context. A fixed per-layer or per-head budget can therefore misallocate memory. Existing methods often treat these two questions separately. Some focus on estimating token importance [Xiao et al., 2023; Li et al., 2024c; Cai et al., 2025; Bui et al., 2025], while others design budget-allocation rules across heads, layers, or modalities [Feng et al., 2024; Qin et al., 2025; Shi et al., 2023; Tu et al., 2024], but often relying on myopic attention statistics.

In this work, we seek a simple unified solution based on learnable retention scores. Building on token retention [Bui et al., 2025], we use lightweight retention gates to predict a scalar future-utility score for each cached KV entry. These scores are trained under a memory constraint to capture whether a token is likely to remain useful for future decoding, rather than merely how much it is attended to at the current step. To make scores comparable across layers and heads, we tie the final scoring projection of all retention gates. This weight sharing calibrates retention scores onto a common scale.

With globally calibrated scores, KV eviction becomes a single ranking problem. Instead of imposing fixed budgets for each layer or head, we maintain one global KV budget and retain the entries with the highest predicted utility across all layers, heads, and modalities. This allows cache capacity to be allocated dynamically: layers and heads that preserve useful long-range information can receive more memory, while those dominated by low-utility or distracting tokens receive less. The resulting policy jointly performs token selection and budget allocation through the same learned retention score.

Empirically, we evaluate our method on long-context language and vision-language benchmarks. Across long-horizon reasoning, multi-turn dialogue, and multimodal understanding tasks, globally calibrated retention substantially reduces KV memory while matching or surpassing full-cache inference. In many cases, selective eviction improves accuracy over the full cache, supporting the view that removing low-utility tokens can improve reasoning rather than simply reduce cost.

Our contributions are summarized as follows:

- We identify attention dilution as a mechanism by which full-cache inference can degrade in long contexts, and show theoretically that preferentially evicting distractors can improve attention quality. We justify geometric retention as a query-agnostic surrogate for future token utility, supported by empirical survival patterns of attended tokens in long-contexts.
- We introduce weight-tied retention gates that learn future-utility scores and calibrate them across layers and heads, enabling direct global comparison of cached KV entries. We propose a global retention-based KV eviction policy that jointly performs token selection and dynamic cache allocation under a single memory budget across layers, heads, and modalities.
- Through experiments on long-context language and vision-language benchmarks, we show that our method improves efficiency and can match or exceed full-cache performance while using substantially less KV memory.

2. Preliminaries

2.1. Self-Attention and KV Eviction

Consider autoregressive generation in a transformer with self-attention. At decoding step t , the KV cache contains all previously generated tokens $C_t = \{1, \dots, t\}$, where each token $i \in C_t$ is associated with a key-value pair $(\mathbf{k}_i, \mathbf{v}_i)$. Given the query $\mathbf{q}_t$ at step t , attention output is

$$z_{t,i} := \frac{\mathbf{q}_t^\top \mathbf{k}_i}{\sqrt{d}}, \quad \alpha_{t,i} := \frac{e^{z_{t,i}}}{\sum_{j \in C_t} e^{z_{t,j}}}, \quad \mathbf{o}_t = \sum_{i \in C_t} \alpha_{t,i} \mathbf{v}_i.$$

As decoding proceeds, the cache grows linearly with t . Consequently, KV memory scales with context length, while the cumulative attention cost over generation scales quadratically [Keles et al., 2023].

A standard approach for improving memory and computation is to restrict the cache to at most M tokens by evicting less important key-value pairs. Under such an eviction policy, the attention becomes

$$\alpha_{t,i}^r = \frac{r_{t,i} e^{z_{t,i}}}{\sum_{j \in C_t} r_{t,j} e^{z_{t,j}}}, \quad \mathbf{o}_t' = \sum_{i \in C_t} \alpha_{t,i}^r \mathbf{v}_i, \quad r_{t,i} \in \{0, 1\}, \quad r_{t,i} \geq r_{t+1,i}. \quad (1)$$

Here, $r_{t,i}$ indicates whether token i is retained in the cache at step t . The monotonicity constraint ensures that once a token is evicted, it cannot re-enter the cache later. The ideal eviction policy solves

$$\min_r \mathcal{L}(\mathbf{o}_t'; \mathbf{o}_t) \quad \text{s.t.} \quad \sum_{i=1}^t r_{t,i} \leq M. \quad (2)$$

That is, we seek a size- M cache whose attention output remains as close as possible to the full-attention output. Solving this combinatorial problem exactly at every decoding step is infeasible, so most prior work relies on heuristic eviction rules [Xiao et al., 2023; Han et al., 2023; Zhang et al., 2023; Li et al., 2024c; Cai et al., 2025; Ghadia et al., 2025].

2.2. Token Retention as a Learnable Surrogate

Bui et al. [2025] relax the discrete variables $r_{t,i} \in \{0, 1\}$ into continuous retention factors by assuming that each token has an intrinsic importance that decays exponentially over time. Under this relaxation,

$$\tilde{\alpha}_{t,i} = \frac{\beta_i^{t-i} e^{z_{t,i}}}{\sum_{j \in C_t} \beta_j^{t-j} e^{z_{t,j}}}, \quad \tilde{\mathbf{o}}_t = \sum_{i \in C_t} \tilde{\alpha}_{t,i} \mathbf{v}_i,$$

where $\beta_i \in [0, 1]$ is a learnable retention score for token i . Larger β_i corresponds to greater long-term importance, while $\beta_i = 1$ recovers standard attention. The score β_i is predicted from the token embedding by a small *retention gate*. These gates are trained so that a student model with retention-gated attention matches a full-cache teacher under a memory budget. Let $p(\cdot | x)$ denote the teacher distribution and $q_\theta(\cdot | x)$ the student distribution with gate parameters θ . The quality loss is

$$\mathcal{L}_{\text{quality}} = D_{\text{KL}}(p \| q_\theta) + \mathbb{E}_{(x,y)}[-\log q_\theta(y | x)]. \quad (3)$$

To enforce the capacity constraint, they introduce

$$\mathcal{L}_{\text{cap}} = \sum_{\ell,h} \sum_{t=1}^T \max\left(0, \sum_{i=1}^t \beta_{\ell,h,i}^{t-i} - M\right), \quad (4)$$

where M is the KV budget for each attention head. The overall objective is

$$\mathcal{L} = \mathcal{L}_{\text{quality}} + \lambda \mathcal{L}_{\text{cap}}. \quad (5)$$

3. Can KV Eviction Improve Long-Context Performance?

In this section, we explain why KV eviction can improve long-context performance through *attention dilution*: full-cache attention spreads mass over many irrelevant tokens, while selective eviction suppresses distractors and concentrates attention on useful context. We further interpret geometric retention as a query-agnostic surrogate for future token utility.

3.1. Attention Dilution as Loss of Useful Mass

Long-context failures can arise even when the relevant evidence is present in the cache [Liu et al., 2024c; Yang et al., 2025a] (see Figure 1a). At a decoding step t , only a small subset $U_t \subseteq C_t$ may be useful for the next prediction, while the remaining tokens act as distractors [Deng et al., 2024]. Since self-attention normalizes over the entire cache, useful tokens must compete with all distractors in the softmax denominator. As the number of distractors grows, the total attention mass assigned to useful tokens can vanish.

To make this precise, define the oracle sparse attention that keeps the same logits on useful tokens but removes all distractors:

$$\alpha_{t,i}^* = \frac{\mathbf{1}\{i \in U_t\} e^{z_{t,i}}}{\sum_{j \in U_t} e^{z_{t,j}}}. \quad (6)$$

This oracle is motivated by the empirical observation that LLMs often perform well in short-context settings but degrade as the context length increases and irrelevant tokens compete for attention [Bansal et al., 2026].

We define the *attention dilution* at step t as the fraction of attention mass assigned to distractors:

$$\delta_t := 1 - \sum_{i \in U_t} \alpha_{t,i}. \quad (7)$$

Equivalently, $\delta_t = \text{TV}(\alpha_t^*, \alpha_t)$. Thus, the dilution quantity is exactly the distance between full-cache attention and the oracle distribution that attends only to useful tokens.

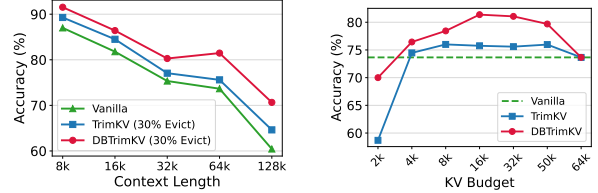
Near-tie distractors force attention dilution. The next result shows that severe dilution is unavoidable when many distractors have logits close to those of the useful tokens.

Proposition 3.1 (Near-tie distractors force dilution). *Fix a decoding step t . Suppose there exist $\Delta \geq 0$ and a subset $D'_t \subseteq C_t \setminus U_t$ such that $z_{t,d} \geq \max_{i \in U_t} z_{t,i} - \Delta$ for all $d \in D'_t$. Then*

$$\delta_t \geq \frac{e^{-\Delta} |D'_t| / |U_t|}{1 + e^{-\Delta} |D'_t| / |U_t|}.$$

Consequently, if $\Delta = O(1)$, $|U_t| = O(1)$, and $|D'_t| \rightarrow \infty$, then $\delta_t \rightarrow 1$ and $\sum_{i \in U_t} \alpha_{t,i} \rightarrow 0$.

The proof is given in Appendix A.1. The proof is given in Appendix A.1. This proposition shows that attention dilution can arise from the cumulative effect of many *competitive* distractors. Although each distractor may receive only a small amount of attention individually, their combined contribution to the softmax denominator can absorb a substantial fraction of the total attention mass, hence diluting the information carried by useful tokens.



(a) Varying Context Length (b) Varying KV Budget

Figure 1: a): Multi-key, -value, -needle NIAH accuracy across increasing context lengths. **b):** Accuracy at 64k context and varying budgets.

Eviction mitigates dilution. We show that selective eviction can mitigate the dilution effect defined in Eq. (7). Let $r_{t,i} \in [0, 1]$ be a general retention weight and $\alpha_{t,i}^r$ be the retention-gated attention in Eq (1). Here, hard eviction is the special case $r_{t,i} \in \{0, 1\}$ and geometric retention is $r_{t,i} = \beta_i^{t-i}$.

Corollary 3.2 (Preferential retention reduces dilution). *For any decoding step t , we have*

$$\delta_t^r = \frac{(\rho_D / \rho_U) \delta_t}{(1 - \delta_t) + (\rho_D / \rho_U) \delta_t}, \quad \text{where} \quad \rho_U = \frac{\sum_{i \in U_t} r_{t,i} e^{z_{t,i}}}{\sum_{i \in U_t} e^{z_{t,i}}}, \quad \rho_D = \frac{\sum_{i \notin U_t} r_{t,i} e^{z_{t,i}}}{\sum_{i \notin U_t} e^{z_{t,i}}}.$$

Consequently, if $\rho_D \leq \rho_U$, then $\delta_t^r \leq \delta_t$. If the ratio $\rho_D / \rho_U \rightarrow 0$, then $\delta^r \rightarrow 0$.

Corollary 3.2 provides an intuition for why eviction can improve long-context behavior. Its condition, $\rho_U \geq \rho_D$, only requires useful tokens to be retained at a higher logit-weighted average rate than distractors. Thus, any retention rule that suppresses distractor mass more than useful-token mass reduces distractor-induced dilution. From this perspective, KV eviction is not merely an efficient approximation to full-cache attention: when irrelevant context dilutes attention, selective eviction can serve as a corrective mechanism. The empirical results for learnable eviction methods in Figure 1 are consistent with this prediction, as the best-performing model is not the full-cache model.

3.2. Geometric Retention as Query-Agnostic Future Utility

We now justify the geometric retention form $r_{t,i} = \beta_i^{t-i}$ used by retention-gated attention. The key intuition is that token importance is both sparse and local. Many tokens are important for the current query or nearby queries, but their utility fades quickly once the generation moves to a different subproblem, entity, or topic. Other tokens remain useful for much longer. Thus, the quantity we want for KV eviction is not only the current attention score of token i , but its future persistence: how likely it is to keep receiving non-negligible attention as decoding continues.

At decoding step t , we view compression as a one-shot decision over the old cache $C_t = \{1, \dots, t\}$. Future tokens are not part of this decision and can be handled by later compression rounds. For an old token $i \leq t$, define its cumulative future utility as

$$\bar{G}_i(t) := \sum_{s=t+1}^T w_{t,s} \Pr(i \in U_s^{(t)} \mid \mathcal{F}_t), \quad (8)$$

where $w_{t,s} \geq 0$ are horizon weights and $U_s^{(t)} \subseteq C_t$ denotes the old cached tokens that remain useful at future step s . Here, $\mathcal{F}_t$ summarizes information available at step t .

To reason about this probability, consider a fixed attention head and approximate future query-key compatibility in a low-dimensional query state:

$$z_{s,i} = \mathbf{x}_s^\top (\mathbf{W}_Q^\top \mathbf{W}_K) \mathbf{x}_i \approx \mathbf{r}_s^\top \mathbf{c}_i, \quad \mathbf{r}_s, \mathbf{c}_i \in \mathbb{R}^m, \quad m \ll d.$$

Here $\mathbf{r}_s$ is the future query state and $\mathbf{c}_i$ is a fixed compatibility vector for token i . For a query state $\mathbf{r}$, let

$$\Gamma_{K,t}(\mathbf{r}) := K\text{-th largest value in } \{\mathbf{r}^\top \mathbf{c}_j : j \leq t\}$$

be the old-cache top- K threshold. Token i is immediately useful at step s if $\mathbf{r}^\top \mathbf{c}_i \geq \Gamma_{K,t}(\mathbf{r})$.

Immediate top- K membership, however, is too strict for retention. A token may temporarily fall below the top- K boundary but may become useful again when generation returns to the same entity,

instruction, document, or topic. We therefore define a relaxed survival region

$$\mathcal{S}_{i,t}^{(K)} = \left\{ \mathbf{r} : \mathbf{r}^\top \mathbf{c}_i \geq \Gamma_{K,t}(\mathbf{r}) - \Delta_{i,t}(\mathbf{r}) \right\}, \quad \Delta_{i,t}(\mathbf{r}) \geq 0.$$

The slack $\Delta_{i,t}(\mathbf{r})$ allows token i to remain retention-worthy even when it is not currently in the hard top- K . It is token dependent: local tokens may admit only a small slack, whereas globally useful tokens, such as topic-summary tokens [Mu et al., 2023], delimiter tokens, or other persistent structural tokens, may tolerate a larger drop below the top- K boundary. For a fixed compression time t and a future step $s > t$, define

$$U_s^{(t)} := \left\{ i \leq t : \mathbf{r}_u \in \mathcal{S}_{i,t}^{(K)} \text{ for all } u = t+1, \dots, s \right\}.$$

This definition separates immediate usefulness from future retention-worthiness. The hard top- K region captures tokens that are among the strongest competitors for the current query state, while the relaxed region captures tokens that remain relevant to become useful again. Since KV eviction is monotone, a removed token cannot re-enter the cache; retention should therefore estimate whether future query states are likely to remain inside this survival region.

The following result justifies the geometric decay of the retention under these definitions.

Theorem 3.3 (Geometric decay of retention (Informal)). *Assume that, within an attention head, future query states $(\mathbf{r}_s)_{s>t}$ evolve according to stable dynamics. If, whenever the query state is inside token i ’s relaxed top- K region $\mathcal{S}_{i,t}^{(K)}$, there is probability at least $\epsilon_i > 0$ of exiting this region within the next b_i decoding steps, then there exists $A_i(t)$ such that*

$$\bar{G}_i(t) \leq A_i(t) \sum_{s=t+1}^T w_{t,s} \beta_i^{s-t}, \quad \beta_i = (1 - \epsilon_i)^{1/b_i} \in (0, 1).$$

The parameter β_i therefore summarizes the persistence of token i : short-lived local tokens have small β_i , while globally useful or structural tokens have β_i close to one. We give the formal statement and proof in Appendix A.2.

Empirically, token persistence exhibits this fast-decay structure under full-cache inference. We run Qwen3-VL-4B on 98 long MMDU multimodal multi-turn dialogues with interleaved text and images, prefilled to $T = 16,384$ tokens. For each decoder layer and head, we record which past tokens are selected by each query. A token is counted as alive at horizon L if it is selected by some query at least L positions after its birth. Figure 2 shows that survival drops rapidly with L : for the shown head, only 13.8% of tokens survive to $L = 4096$ under top- $K = 256$, and only 3.2% survive to $L = 16k$. Even under the more lenient 99%-mass criterion, the head loses 60% of mass-relevant tokens by $L = 4096$. Vision tokens, which dominate the multimodal cache, fade at a similar rate as text tokens. In this regime, retaining every old token can dilute attention mass, while evicting tokens whose future utility has already decayed can recover capacity for tokens that remain relevant.

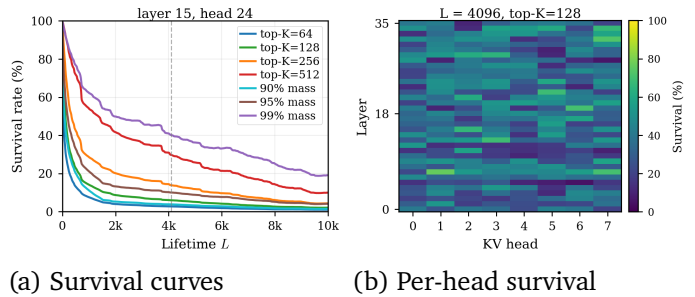


Figure 2: a) Fraction of tokens whose attention persists to horizon L for layer 15, head 24. **b)** Per-layer, per-head survival at $L = 4096$ and top- $K = 128$.

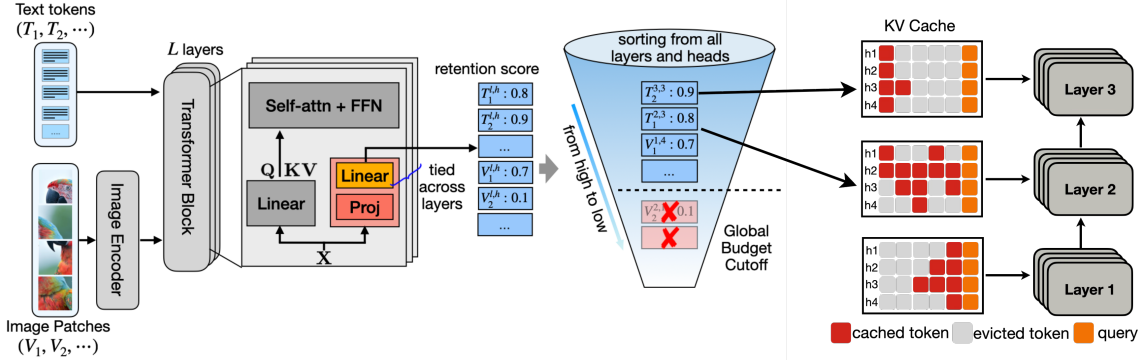


Figure 3: Prefill/compression with our method. A per-layer, per-head retention gate assigns each text/visual token a scalar score; the gate’s final projection is tied across layers to normalize scores. Tokens from all layers, heads, and modalities are then pooled and globally ranked by retention, replacing fixed per-layer/per-head KV budgets.

These results support geometric retention as a simple surrogate for future token persistence. In practice, we do not explicitly estimate the exit parameters (b_i, ϵ_i) . Instead, the retention gate predicts β_i directly from the token representation, allowing the model to learn which tokens should decay quickly and which should persist. To avoid recomputing β_i at every compression step, we estimate it only once when the token enters the cache. The resulting retention weight then follows the geometric form

$$r_{t,i} = \beta_i^{t-i},$$

which can be interpreted as the predicted probability that token i remains useful at step t given the information available when it was first cached.

4. Global Token Retention via Weight-Tied Gates

The per-head survival heatmap in Figure 2b shows strong heterogeneity: a small number of heads preserve long-range tokens, while many heads quickly lose them. This raises a key question: "how should a limited KV budget be allocated across layers and heads?". Existing methods typically use fixed per-layer or per-head budgets [Bui et al., 2025], or adaptively allocate budgets using a myopic attention heuristics [Feng et al., 2024]. A natural alternative is to rank tokens by retention, but standard retention gates are trained independently across layers and heads, so their scores are not directly comparable.

To address this, we introduce *global* KV eviction via weight-tied retention gates. Specifically, we use per-layer, per-head gates whose *final scoring projection is shared* across all layers and heads, placing retention scores on a common scale. This allows all KV entries to be ranked globally under a single cache budget, replacing hand-designed per-layer or per-head allocations with a unified eviction rule.

4.1. Architecture and Training

Weight-tied retention gates. Consider a transformer with L layers and H attention heads per layer. For each token at position t , layer ℓ , and head h , we predict a retention coefficient

$$\beta_{\ell,h,t} = g_{\ell,h}(\mathbf{x}_t) \in [0, 1],$$

where $\mathbf{x}_t$ is the token embedding. Our main design choice is the parameterization of $g_{\ell,h}$. Each gate first computes a head-specific embedding, but the final scalar score is produced by a *shared* projection:

$$g_{\ell,h}(\mathbf{x}) = \sigma\left(\mathbf{w}_g^\top \text{Proj}_{\ell,h}(\mathbf{x}) + b_g\right),$$

where $\text{Proj}_{\ell,h}$ is layer/head-specific, while $(\mathbf{w}_g, b_g)$ is tied across all layers and heads. This shared readout calibrates retention scores globally: a score produced in one head has the same meaning as the same score produced in another. Without such tying, retention values are only locally meaningful and cannot reliably support global eviction.

Training. We follow the same training procedure as [Bui et al., 2025] described in Section 2.2 but replace the local capacity constraint by a global counterpart

$$\mathcal{L}_{\text{cap}} = \sum_{t=1}^T \max \left(0, \sum_{\ell,h} \sum_{i=1}^t \beta_{\ell,h,i}^{t-i} - M_{\text{global}} \right),$$

where M_{global} is the target global KV budget. We only train retention gates and freeze LLM weights.

4.2. Global KV Eviction

At inference time, we use the learned retention coefficients to rank all cached KV entries globally. For each cached token (ℓ, h, i) present at decoding step t , we assign the retention score

$$\tilde{G}_{\ell,h,i}(t) = \sum_{s=t+1}^T \beta_{\ell,h,i}^{s-i} = \beta_{\ell,h,i}^{t+1-i} \frac{1 - \beta_{\ell,h,i}^{T-t}}{1 - \beta_{\ell,h,i}}, \quad \text{for } \beta_{\ell,h,i} < 1. \quad (9)$$

Here, we choose the simple horizon weight $w_{t,s} = 1$. Thus, $\tilde{G}_{\ell,h,i}(t)$ aggregates the predicted future utility of token (ℓ, h, i) over the remaining decoding horizon $T - t$. In the one-step case $T - t = 1$, it reduces to the myopic score $\tilde{G}_{\ell,h,i}(t) = \beta_{\ell,h,i}^{t+1-i}$, used in [Bui et al., 2025]. More generally, for $\beta_{\ell,h,i} \in [0, 1)$ and $T - t \rightarrow \infty$, then $\tilde{G}_{\ell,h,i}(t) \rightarrow \frac{\beta_{\ell,h,i}^{t+1-i}}{1 - \beta_{\ell,h,i}}$.

The lookahead horizon $T - t$ therefore governs the trade-off between recency and retention. Recency enters through the factor β_i^{t+1-i} , while longer horizons increasingly favor tokens with larger retention parameters β_i . This softens the effective local sliding-window and geometric decay bias and allows older, slowly decaying tokens to remain competitive.

Our eviction rule is simple now: retain the M_{global} tokens with the largest scores $\tilde{G}_{\ell,h,i}(t)$ across all layers and heads. Unlike existing budget-allocation methods, this requires no predefined per-layer or per-head budgets. Instead, capacity is assigned automatically by a unified retention ranking.

Implementation details. To support variable cache sizes induced by dynamic, head-specific budgets, we use a paged-attention layout, similar to Kwon et al. [2023]. Specifically, KV entries are stored in fixed-size pages, and each head maintains a block table for its currently active pages. Attention is then computed with variable-length kernels [Dao, 2023] using the resulting per-head sequence lengths. This allows each head to maintain a variable-length logical KV sequence without requiring contiguous physical storage or kernel specialization. An example is provided in Figure 4.

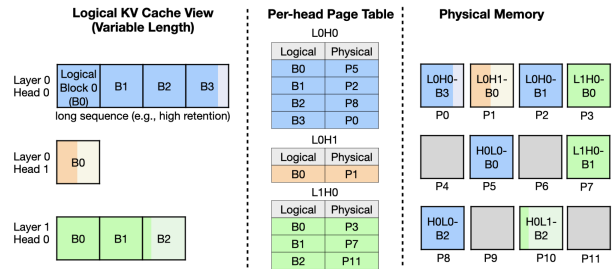


Figure 4: PagedAttention with head-specific variable-length KV caches. Each layer-head maintains a logical KV sequence of variable length, stored as fixed-size blocks.

Method	128 Visual Tokens				64 Visual Tokens				32 Visual Tokens			
	GQA	VQAText	MME	Avg. (%)	GQA	VQAText	MME	Avg. (%)	GQA	VQAText	MME	Avg. (%)
Vanilla*	61.9	58.2	1506.5	100.0	61.9	58.2	1506.5	100.0	61.9	58.2	1506.5	100.0
FastV*	-	-	-	-	46.0	51.6	973.5	75.9	-	-	-	-
DART*	57.9	56.3	1408.7	94.6	54.7	54.7	1365.1	91.0	52.9	52.2	1273.3	86.6
PruMerge*	58.2	54.0	1408.1	93.4	55.4	52.0	1316.8	88.8	52.9	49.2	1236.6	84.0
VisionZip*	57.6	56.9	1436.9	95.4	55.1	55.5	1365.2	91.7	51.8	53.1	1251.2	86.0
CDPruner*	59.9	56.2	1431.4	96.1	58.6	55.3	1415.1	94.5	57.0	53.2	1373.0	91.5
DivPrune*	59.4	55.9	1405.1	95.1	57.5	54.5	1334.7	91.7	54.9	52.9	1284.9	88.3
TrimKV ^o	61.2	57.7	1490.5	99.9	61.2	57.3	1490.5	99.7	61.1	56.8	1490.5	99.4
DBTrimKV ^o	61.2	57.7	1490.5	99.9	61.1	57.3	1490.5	99.7	61.0	57.0	1490.5	99.4

Table 1: Performance of competing method on LLaVA-1.5-7B and short-form question answering tasks. The symbol * indicates that the numbers are copied from [Zhang et al., 2025]. Our result is reproduced by running with the LMMs-Eval framework. In the Avg. column, we report the relative performance compared to the corresponding vanilla inference.

5. Experiments

We evaluate our approach across benchmarks covering: (i) long-horizon reasoning, (ii) multi-turn dialogue, (iii) short-form question answering, and (iv) long-context question answering. Throughout, we refer to our method as *Dynamic Budget TrimKV* (DBTrimKV).

5.1. Short-Form Question Answering

We first study short-form visual question answering, where a relatively small number of output tokens must be generated conditioned on an image and a short textual prompt. In this experiment, we compare DBTrimKV and TrimKV against visual token pruning methods that operate exclusively at the prefilling stage and remove only a subset of visual tokens before decoding. To ensure a fair comparison, we fix the total KV budget of DBTrimKV and TrimKV to be equal to the sum of (i) all text tokens in the prompt and (ii) the visual-token budget used by each pruning baseline.

Benchmarks. We evaluate on three standard short-form VQA benchmarks: VQAText [Singh et al., 2019], MME [Fu et al., 2025a], and GQA [Hudson and Manning, 2019]. These benchmarks cover a range of visual reasoning skills, from simple recognition to compositional and relational reasoning, while keeping generation lengths short.

Baselines. We compare our method against recent visual token pruning approaches, including FastV [Chen et al., 2024a], VisionZip [Yang et al., 2025b], DART [Wen et al., 2025], PruMerge [Shang et al., 2025], DivPrune [Alvar et al., 2025], and CDPruner [Zhang et al., 2025]. All these methods reduce memory usage by selectively pruning visual tokens during prefilling, while leaving the textual KV cache intact throughout decoding.

Implementation details. For this experiment, we use LLaVA-1.5-7B as the base model. We train our method on the LLaVA-Next dataset [Liu et al., 2024b]. Following [Bui et al., 2025], we update only the retention-gate weights while keeping all original model parameters frozen. We set the objective hyperparameter to $\lambda_{\text{cap}} = 1.0$ and the memory capacity to $M_{\text{global}} = 64 * L * H$, where L and H are the number of layers and heads, respectively. Each transformer block is equipped with a retention gate g , implemented as a small MLP with hidden dimension $d_g = 512$. For the retention gate architecture, we implement the Proj as a two-layer MLP. During the training, we tie the final linear layer ($\mathbf{W}, b$) across all layers and heads as described in Section 4. The bias term b is initialized to 18.0 at the start

Budget	Method	Image Reasoning			Video Reasoning				Average (% vs Vanilla)
		MMStar	MathVision _{mini}	MMMUP _{Provision}	VideoMME	VideoMMMU _{adap}	VideoMMMU _{comp}	VideoMathQA _{mcq}	
	Vanilla	71.52	48.68	40.64	54.22	35.67	55.00	36.19	100
1024	SnapKV	51.84	15.13	18.27	49.22	21.00	28.67	20.24	58.03
	R-KV	58.42	24.67	26.24	51.48	22.00	30.33	26.43	68.82
	AdaKV	66.89	32.89	25.61	52.00	25.42	<u>31.67</u>	23.57	73.43
	Ada-Pyramid-KV	66.84	28.29	25.90	53.81	28.96	30.74	23.09	73.63
	TrimKV ^o	70.64	45.72	34.34	54.15	<u>35.00</u>	<u>59.00</u>	35.00	97.02
	DBTrimKV ^o	71.50	52.63	41.1	54.93	37.00	59.33	36.43	103.26
512	SnapKV	51.01	14.80	9.36	49.22	22.74	21.40	18.09	52.60
	R-KV	55.23	20.72	15.54	49.51	20.40	21.84	19.76	57.26
	AdaKV	60.14	17.11	16.99	53.77	22.15	24.67	19.05	59.97
	Ada-Pyramid-KV	60.41	15.46	15.61	52.63	19.73	19.46	17.14	55.68
	TrimKV ^o	70.28	42.11	34.34	54.74	34.00	<u>51.51</u>	32.86	92.85
	DBTrimKV ^o	71.85	51.97	43.87	54.41	35.57	61.33	35.95	103.73
256	SnapKV	50.89	7.89	6.24	50.15	17.11	14.92	14.52	44.35
	R-KV	53.01	16.12	12.37	51.26	14.38	17.27	18.10	50.56
	AdaKV	53.30	8.55	8.27	52.15	17.41	18.77	14.76	47.48
	Ada-Pyramid-KV	52.56	8.22	8.09	50.52	16.90	15.17	15.95	46.07
	TrimKV ^o	65.67	40.79	31.45	53.15	30.67	42.48	28.81	84.83
	DBTrimKV ^o	69.91	51.64	41.85	54.15	39.00	57.00	32.86	101.49
128	SnapKV	40.43	1.64	2.66	46.15	14.58	13.06	11.90	35.58
	R-KV	50.04	8.22	4.51	46.56	11.38	11.03	14.29	39.32
	AdaKV	40.89	2.96	4.14	47.44	16.38	9.59	13.33	37.30
	Ada-Pyramid-KV	41.60	3.62	4.05	45.78	15.99	13.57	12.86	37.86
	TrimKV ^o	62.29	27.30	24.34	52.55	22.90	28.38	22.14	68.13
	DBTrimKV ^o	68.69	47.04	40.41	54.85	34.33	57.00	34.29	98.27
64	TrimKV ^o	<u>54.12</u>	<u>12.17</u>	<u>13.24</u>	49.74	<u>18.40</u>	<u>18.18</u>	19.52	51.94
	DBTrimKV ^o	65.44	33.88	32.49	54.15	28.67	42.67	25.71	81.42

Table 2: Results across image and video reasoning benchmarks. The best eviction methods are shown in **bold**, and the second-best are underlined. Performances that exceed vanilla inference are highlighted in **red**. The superscript ^o denotes a trainable KV eviction method.

of training. During inference, we use a default lookahead horizon $T - t = 2$ for every eviction time t .

Results. As shown in Table 1, both DBTrimKV and TrimKV consistently outperform visual token pruning baselines and match vanilla performance under all memory budgets. This gap highlights a key limitation of prefilling-only visual pruning: by constraining memory exclusively on the vision side, these methods cannot adapt to how different heads and layers actually utilize visual versus textual information during decoding. In contrast, DBTrimKV and TrimKV compress the KV cache jointly over both vision and text tokens and across all layers and heads. This flexibility allows the model to retain more visual tokens in heads that encode rich visual semantics, while aggressively trimming tokens, visual or textual, that contribute less to downstream prediction.

5.2. Long-Horizon Reasoning

Real VLM applications require long-context, long-horizon generation where KV pressure accumulates over many decoding steps. In this regime, eviction must preserve long-lived evidence, and suboptimal allocation compounds over time. We therefore evaluate on long-horizon reasoning benchmarks that amplify these effects through both longer contexts and longer generations.

Benchmarks. We consider three image-based benchmarks, including MMStar [Chen et al., 2024b], MathVision_{mini} [Wang et al., 2024], and MMMUP_{Provision} [Yue et al., 2025], and three video-based benchmarks, such as VideoMME [Fu et al., 2025b], VideoMathQA_{mcq} [Rasheed et al., 2025], and VideoMMMU [Hu et al., 2025] (adaptation and comprehension subsets). For a pure text reasoning, we evaluate on AIME24 [Art of Problem Solving, 2024], GSM8K [Cobbe et al., 2021], and MATH-

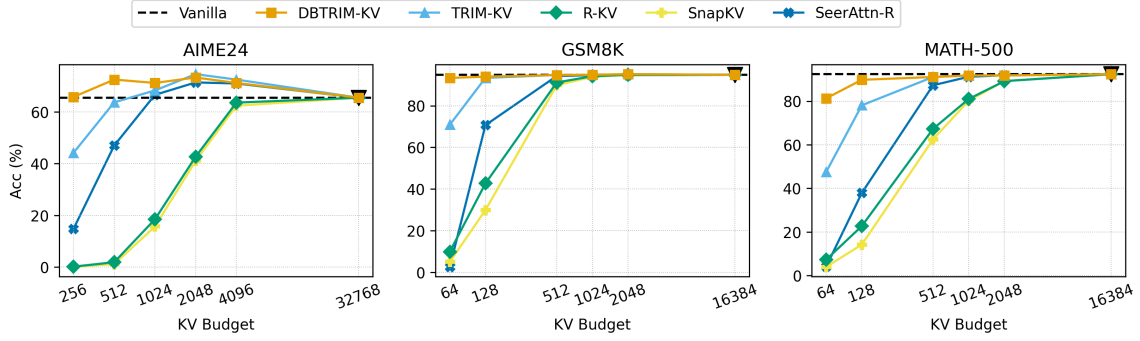


Figure 5: Long-reasoning performance of pure language model (Qwen3-4B).

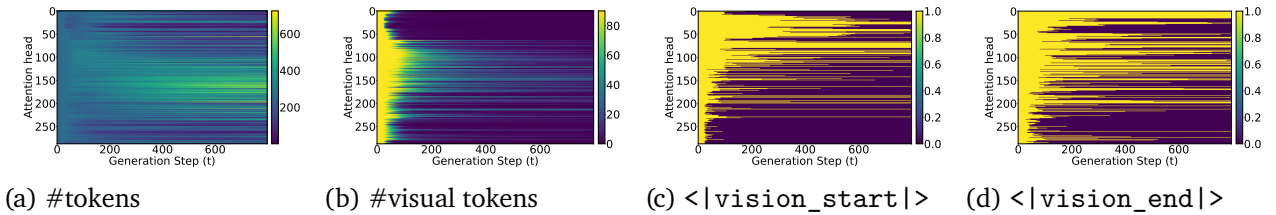


Figure 6: Visualization of KV-cache usage across attention heads for a sample in the MathVision_{mini} dataset. The y -axis indexes the attention heads sequentially from layer 1 to layer 36, with each layer containing 8 KV heads. Figures 6a, 6b, 6c, and 6d show the number of total tokens, visual tokens, $\langle |vision_start| \rangle$, and $\langle |vision_end| \rangle$ tokens that are active in the KV cache, respectively.

500 [Hendrycks et al., 2021], similar to [Bui et al., 2025]. Unless otherwise specified, we use greedy decoding and report accuracy as the evaluation metric. We adopt LMMs-Eval [Zhang et al., 2024a] as the standard framework for evaluating different methods on VLM benchmarks. Results are reported using a global budget of $M \times (\#layers) \times (\#heads)$ for a fair comparison with fixed-budget methods.

Baselines. We compare against strong KV-eviction methods for long-context decoding, including SnapKV [Li et al., 2024c], AdaKV [Feng et al., 2024], Ada-Pyramid-KV [Cai et al., 2024], R-KV [Cai et al., 2025], and TrimKV [Bui et al., 2025]. Among these, AdaKV and Ada-Pyramid-KV extend SnapKV with dynamic budget allocation. For pure-text reasoning, we also include SeerAttn-R [Gao et al., 2025], a state-of-the-art learnable KV retrieval method tailored to reasoning-oriented language models.

Implementation details. We use Qwen3-VL-8B-Thinking as a strong visual reasoning model. To train the retention gates, we use a mixture of multiple training datasets, including M4Instruct [Li et al., 2024a], Academic Video [Zhang et al., 2024c], R1-OneVision [Yang et al., 2025c], and OpenR1-Math-220k [Hugging Face], to cover diverse training scenarios. Since the retention gates are lightweight, comprising only 0.43% of the total VLM parameters, we use a subset of each dataset for training. For the pure text experiment, we mimic the settings from [Bui et al., 2025] and train on Qwen3-4B on OpenR1-Math-220K [Hugging Face] for fair comparisons. Other settings are similar to Section 5.1.

Quantitative results. Table 2 reports the performance of different KV eviction methods on image and video reasoning benchmarks. Figure 5 shows the Pareto fronts of different methods under varying KV budgets in pure text reasoning settings. It can be seen that DBTrimKV achieves state-of-the-art performance in all settings compared to other KV eviction baselines, with particularly large gains in low-budget regimes.

Method	512 Budget			256 Budget			128 Budget		
	Accuracy	Overall	% vs Vanilla	Accuracy	Overall	% vs Vanilla	Accuracy	Overall	% vs Vanilla
Vanilla	3.37	3.57	100	3.37	3.57	100	3.37	3.57	100
SnapKV	3.07	3.46	96.70	2.54	2.88	80.51	2.01	2.29	64.00
R-KV	2.76	3.00	84.00	2.42	2.66	74.45	2.02	2.17	60.77
AdaKV	2.89	3.24	90.79	2.56	2.88	80.68	2.03	2.25	62.98
TrimKV 	<u>3.46</u>	<u>3.73</u>	<u>104.51</u>	3.16	3.56	99.65	3.05	3.35	93.61
DBTrimKV 	3.79	4.09	114.46	3.64	3.87	108.33	3.43	3.72	104.10

Table 3: Comparison of different methods across selected metrics under varying token budgets.

Comparisons between fixed-budget and dynamic-budget methods (DBTrimKV vs. TrimKV and AdaKV/Ada-Pyramid-KV vs. SnapKV) show that dynamically allocating the KV budget generally yields better performance than using a fixed allocation, with the performance gap widening as the overall budget decreases. Notably, DBTrimKV with budgets of 1024, 512, and even 256 surpasses vanilla full-cache inference with up to 3.75%. This indicates that, beyond improving memory and computational efficiency, token retention methods can also potentially enhance model performance relative to standard full-cache decoding. The gains are especially pronounced on challenging tasks that require long-form generation, such as AIME24, MathVision, MMMUPro, and VideoMMMU subsets. In contrast, on tasks involving shorter reasoning, such as GSM8K, MATH-500, MMStar, and VideoMME, the performance differences among KV eviction methods are smaller.

Qualitative results. Figure 6 illustrates DBTrimKV’s highly dynamic and specialized KV cache allocation on a MathVision_{mini} example. The total tokens (Figure 6a) is aggressively compressed and concentrated into a sparse subset of heads, frequently in the mid-layers, suggesting the model selectively preserves context in the specific heads responsible for integrating semantic concepts and synthesizing information. For visual tokens (Figure 6b), dense context is retained early to interpret the image, but shrinks markedly as decoding shifts toward text-centric reasoning. Interestingly, while fine-grained visual patches are heavily evicted, the model persistently retains the boundary tokens `<|vision_start|>` (Figure 6c) and `<|vision_end|>` (Figure 6d) across numerous heads throughout generation. This suggests these tokens act as compressed structural anchors, grounding the model’s reasoning without the memory cost of full visual details. Further visualizations demonstrating DBTrimKV’s ability to isolate task-relevant visual tokens while aggressively discarding background distractors are provided in Appendix C.5.

5.3. Multi-turn Dialogue

We evaluate on multi-turn visual dialogue using MMDU [Liu et al., 2024e] to assess KV eviction under interactive, open-ended settings. Unlike single-shot QA, multi-turn dialogue requires the model to retain and selectively reuse information introduced many turns earlier, while accommodating new, potentially unexpected questions at each turn. In this experiment, we use Qwen3-VL-4B-Instruct as the base model and train on the MMDU-45K dataset [Liu et al., 2024e]. For evaluation, we adopt Gemini3 [Team et al., 2023; Comanici et al., 2025] as an LLM-based judge, following the MMDU protocol. Other settings are similar to Section 5.2.

Table 3 shows that DBTrimKV and TrimKV outperform heuristic eviction baselines by a wide margin, with the advantage widening as the KV budget shrinks. Notably, DBTrimKV at all budgets outperforms vanilla inference with full cache (by 14% at $M = 512$). These results underscore the value of dynamic allocation: DBTrimKV consistently improves over its fixed-budget counterpart (TrimKV), and AdaKV

similarly improves over SnapKV, indicating that adaptive capacity allocation is especially crucial for long, interactive dialogue applications.

5.4. Runtime and Ablations

We conduct ablation studies on the MathVision dataset to isolate the impact of our gate design.

No weight tying. Table 4 removes weight tying in the final linear layer of the retention gates. Weight tying yields a substantial improvement, especially in the low-budget regime, consistent with its role in making retention scores comparable across layers and heads.

Retention gate inputs. We also test an alternative gate that takes the concatenation of the key and value vectors as input. As shown in Table 5, using the token contextual embedding $\mathbf{x}_t$ as input performs better, particularly under tight budgets, suggesting that $\mathbf{x}_t$ provides a stronger long-horizon utility signal than the projected KV representations alone.

Runtime. Figure 7 reports total decoding time in seconds as the number of generated tokens increases. Evaluated methods include Vanilla (Full Cache), DBTrimKV, and TrimKV. We fix batch size at 16, context length at 16384 and KV budget at 256. DBTrimKV with PagedAttention remains highly efficient. DBTrimKV adds a small overhead compared to TrimKV due to variable-length, head-specific cache handling; however, it still outperforms vanilla inference at scale. Overall, DBTrimKV is practical for large-batch deployment, with PagedAttention enabling efficient dynamic budgeting. We expect the speedups over vanilla to grow with longer contexts and longer generations, where KV bandwidth becomes the dominant bottleneck. More details are provided in Appendix C.4.

Method/Budget	256	512
Vanilla	48.68	
TrimKV	40.79	42.14
DBTrimKV w/o tying	40.13	44.74
DBTrimKV	51.64	51.67

Table 4: Weight tying ablation.

Input/Budget	256	1024
DBTrimKV $-g_{\ell,h}([\mathbf{k}_t \mathbf{v}_t])$	43.75	46.71
DBTrimKV $-g_{\ell,h}(\mathbf{x}_t)$	51.64	52.63

Table 5: Input ablation.

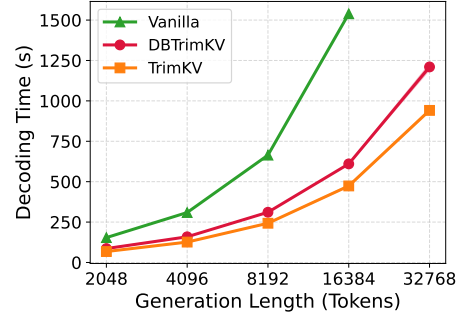


Figure 7: Efficiency scaling with generation length.

6. Related Work

KV cache compression. Since the advent of long-context LMs and VLMs [Liu et al., 2025a], KV-cache compression has become a central efficiency challenge [Li et al., 2024b]. Early efforts primarily target the prefill stage and can be grouped into three main directions: (i) vector compression and quantization [Hooper et al., 2024; Liu et al., 2024d; Yue et al., 2024; Sun et al., 2024], (ii) token retrieval [Tang et al., 2024; Liu et al., 2024a; Gao et al., 2025], and (iii) token eviction or merging [Zhang et al., 2023; Nawrot et al., 2024; Zhang et al., 2024d; Qin et al., 2025; Wang et al., 2025; Liu et al., 2025b; Park et al., 2025]. While effective in many settings, these approaches often incur nontrivial systems overhead (e.g., compression, selection, or data movement), making them less suitable for long-horizon generation tasks such as long reasoning, interactive use, or streaming applications. In these scenarios, simple token-eviction heuristics [Xiao et al., 2023; Li et al., 2024c; Cai et al., 2025] are typically preferred due to their low overhead, at the cost of potential performance degradation. Recent work [Bui et al., 2025] proposes token retention as an effective learnable alternative for driving eviction policies. However, most studies on long-horizon efficiency focus on text-only LLMs, and the behavior and effectiveness

of these methods in multimodal settings remain underexplored. We bridge this gap by providing an extensive evaluation of KV-cache compression strategies, including retention-based eviction, on VLM benchmarks.

Dynamic budget allocation for KV cache. Beyond selecting *which* tokens to evict, a complementary line of work studies how KV-cache *capacity* should be allocated across layers, heads, or modalities. For text-only LMs, AdaKV [Feng et al., 2024] adapts per-layer KV sizes using attention statistics, PyramidKV [Cai et al., 2024] uses a hand-crafted pyramid that favors lower layers, CAKE [Qin et al., 2025] couples token clustering with layer-wise budget control, and ZigZagKV [Zhong et al., 2025] alternates dense and sparse layers to trade memory for accuracy. Dynamic budgeting has also been explored for VLMs: VL-KV [Tu et al., 2024] introduces modality-aware KV compression, while MEDA [Wan et al., 2025] allocates attention and memory differently across visual and textual tokens or visual frame as in [Li et al., 2025]. Despite improved utilization over uniform budgets, these methods typically rely on heuristics or separate, locally enforced budgets. In contrast, we eliminate per-layer/head/modality budgets and enforce a single *global* KV constraint, using learned retention scores as the sole signal to jointly drive eviction and allocation.

Visual token pruning. In parallel with KV cache compression, a separate line of work focuses on visual token pruning. Recent approaches, such as VisionZip [Yang et al., 2025b] and FastV [Chen et al., 2024a], prune tokens using pretrained attention maps, while DivPrune [Alvar et al., 2025] and CDPruner [Zhang et al., 2025] enforce diversity within the selected vision tokens and SparseVLM [Zhang et al., 2024b] scores patches via text-to-vision attention. However, these methods operate primarily on vision tokens and often rely on unimodal or local attention signals for pruning. In contrast, our approach acts directly on the KV cache and jointly manages both vision and text tokens under a unified memory budget, allocating capacity across heads and layers based on downstream utility.

7. Conclusion

We introduced DBTrimKV, a globally calibrated KV eviction method for long-context LMs and VLMs. DBTrimKV learns retention scores that estimate the future utility of cached tokens and uses a single global budget to allocate memory dynamically across layers, heads, and modalities. Our analysis and experiments show that selective eviction can reduce attention dilution, improve reasoning over relevant context, and substantially lower KV memory. Across language, vision-language, and multi-turn dialogue benchmarks, DBTrimKV consistently outperforms prior eviction baselines and can match or exceed full-cache inference with much less memory. Beyond improving inference efficiency, our results suggest a broader view of KV eviction: selective forgetting can act as a form of attention regularization that suppresses distractors and sharpens reasoning over relevant context. We hope this work motivates future research on learned memory management for long-context models, including adaptive retention objectives, retrieval-aware cache policies, and training-time integration of eviction mechanisms into large-scale foundation models.

References

- Saeed Ranjbar Alvar, Gursimran Singh, Mohammad Akbari, and Yong Zhang. Divprune: Diversity-based visual token pruning for large multimodal models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 9392–9401, 2025.
- Art of Problem Solving. AIME problems and solutions, 2024. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.

- Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. *arXiv preprint arXiv:2412.15204*, 2024.
- Rachit Bansal, Aston Zhang, Rishabh Tiwari, Lovish Madaan, Sai Surya Duvvuri, Fnu Devvrit, David Brandfonbrener, David Alvarez-Melis, Prajjwal Bhargava, Mihir Kale, and Samy Jelassi. Let’s (not) just put things in context: Test-time training for long-context LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Ngoc Bui, Shubham Sharma, Simran Lamba, Saumitra Mishra, and Rex Ying. Cache what lasts: Token retention for memory-bounded kv cache in llms. *arXiv preprint arXiv:2512.03324*, 2025.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.
- Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, et al. R-kv: Redundancy-aware kv cache compression for training-free reasoning models acceleration. *arXiv preprint arXiv:2505.24133*, 2025.
- Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35. Springer, 2024a.
- Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, et al. Are we on the right way for evaluating large vision-language models? *Advances in Neural Information Processing Systems*, 37:27056–27087, 2024b.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691*, 2023.
- Yichuan Deng, Zhao Song, Jing Xiong, and Chiwun Yang. How sparse attention approximates exact attention? your attention is naturally n^C -sparse. *arXiv preprint arXiv:2404.02690*, 2024.
- Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *arXiv preprint arXiv:2407.11550*, 2024.
- Chaoyou Fu, Peixian Chen, Yunhang Shen, Yulei Qin, Mengdan Zhang, Xu Lin, Jinrui Yang, Xiawu Zheng, Ke Li, Xing Sun, et al. Mme: A comprehensive evaluation benchmark for multimodal large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025a.
- Chaoyou Fu, Yuhan Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation

- benchmark of multi-modal llms in video analysis. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 24108–24118, 2025b.
- Yizhao Gao, Shuming Guo, Shijie Cao, Yuqing Xia, Yu Cheng, Lei Wang, Lingxiao Ma, Yutao Sun, Tianzhu Ye, Li Dong, et al. Seerattention-r: Sparse attention adaptation for long reasoning. *arXiv preprint arXiv:2506.08889*, 2025.
- Ravi Ghadia, Avinash Kumar, Gaurav Jain, Prashant Nair, and Poulami Das. Dialogue without limits: Constant-sized kv caches for extended responses in llms. *arXiv preprint arXiv:2503.00979*, 2025.
- Chi Han, Qifan Wang, Hao Peng, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. Lm-infinite: Zero-shot extreme length generalization for large language models. *arXiv preprint arXiv:2308.16137*, 2023.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems*, 37:1270–1303, 2024.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024.
- Kairui Hu, Penghao Wu, Fanyi Pu, Wang Xiao, Yuanhan Zhang, Xiang Yue, Bo Li, and Ziwei Liu. Video-mmmu: Evaluating knowledge acquisition from multi-discipline professional videos. *arXiv preprint arXiv:2501.13826*, 2025.
- Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin. Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*, 2025a.
- Yuxiang Huang, Binhang Yuan, Xu Han, Chaojun Xiao, and Zhiyuan Liu. Locret: Enhancing eviction in long-context LLM inference with trained retaining heads on consumer-grade devices. *Transactions on Machine Learning Research*, 2025b. ISSN 2835-8856. J2C Certification.
- Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019.
- Hugging Face. Open r1: A fully open reproduction of deepseek-r1. URL <https://github.com/huggingface/open-r1>.
- Feyza Duman Keles, Pruthuvi Mahesakya Wijewardena, and Chinmay Hegde. On the computational complexity of self-attention. In *International conference on algorithmic learning theory*, pages 597–619. PMLR, 2023.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

- Feng Li, Renrui Zhang, Hao Zhang, Yuanhan Zhang, Bo Li, Wei Li, Zejun Ma, and Chunyuan Li. Llava-next-interleave: Tackling multi-image, video, and 3d in large multimodal models. *arXiv preprint arXiv:2407.07895*, 2024a.
- Haoyang Li, Yiming Li, Anxin Tian, Tianhao Tang, Zhanchao Xu, Xuejia Chen, Nicole Hu, Wei Dong, Qing Li, and Lei Chen. A survey on large language model acceleration based on kv cache management. *arXiv preprint arXiv:2412.19442*, 2024b.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024c.
- Yulin Li, Haokun Gui, Ziyang Fan, Junjie Wang, Bin Kang, Bin Chen, and Zhuotao Tian. Less is more, but where? dynamic token compression via llm-guided keyframe prior. *arXiv preprint arXiv:2512.06866*, 2025.
- Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, et al. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *arXiv preprint arXiv:2409.10516*, 2024a.
- Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024b. URL <https://llava-vl.github.io/blog/2024-01-30-llava-next>.
- Jiaheng Liu, Dawei Zhu, Zhiqi Bai, Yancheng He, Huanxuan Liao, Haoran Que, Zekun Wang, Chenchen Zhang, Ge Zhang, Jiebin Zhang, et al. A comprehensive survey on long context language modeling. *arXiv preprint arXiv:2503.17407*, 2025a.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024c.
- Xiang Liu, Zhenheng Tang, Hong Chen, Peijie Dong, Zeyu Li, Xiuze Zhou, Bo Li, Xuming Hu, and Xiaowen Chu. Can llms maintain fundamental abilities under kv cache compression? *arXiv preprint arXiv:2502.01941*, 2025b.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024d.
- Ziyu Liu, Tao Chu, Yuhang Zang, Xilin Wei, Xiaoyi Dong, Pan Zhang, Zijian Liang, Yuanjun Xiong, Yu Qiao, Dahua Lin, and Jiaqi Wang. MMDU: A multi-turn multi-image dialog understanding benchmark and instruction-tuning dataset for LVLMS. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024e.
- Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems*, 36:19327–19352, 2023.
- Piotr Nawrot, Adrian Łańcucki, Marcin Chochowski, David Tarjan, and Edoardo M Ponti. Dynamic memory compression: Retrofitting llms for accelerated inference. *arXiv preprint arXiv:2403.09636*, 2024.
- Junyoung Park, Dalton Jones, Matthew J Morse, Raghavv Goel, Mingy Lee, and Chris Lott. Keyd-

- iff: Key similarity-based kv cache eviction for long-context llm inference in resource-constrained environments. *arXiv preprint arXiv:2504.15364*, 2025.
- Ziran Qin, Yuchen Cao, Mingbao Lin, Wen Hu, Shixuan Fan, Ke Cheng, Weiyao Lin, and Jianguo Li. Cake: Cascading and adaptive kv cache eviction with layer preferences. *arXiv preprint arXiv:2503.12491*, 2025.
- Hanoona Rasheed, Abdelrahman Shaker, Anqi Tang, Muhammad Maaz, Ming-Hsuan Yang, Salman Khan, and Fahad Shahbaz Khan. Videomathqa: Benchmarking mathematical reasoning via multi-modal understanding in videos. *arXiv preprint arXiv:2506.05349*, 2025.
- Ranjan Sapkota, Yang Cao, Konstantinos I Roumeliotis, and Manoj Karkee. Vision-language-action models: Concepts, progress, applications and challenges. *arXiv preprint arXiv:2505.04769*, 2025.
- Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22857–22867, 2025.
- Xiaohang Shi, Sheng Zhang, Jie Wu, Ning Chen, Ke Cheng, Yu Liang, and Sanglu Lu. Adapypramid: Adaptive pyramid for accelerating high-resolution object detection on edge devices. *IEEE Transactions on Mobile Computing*, 23(8):8208–8224, 2023.
- Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8317–8326, 2019.
- Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. Shadowkv: Kv cache in shadows for high-throughput long-context llm inference. *arXiv preprint arXiv:2410.21465*, 2024.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. *arXiv preprint arXiv:2406.10774*, 2024.
- Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Dezhan Tu, Danylo Vashchilenko, Yuzhe Lu, and Panpan Xu. Vl-cache: Sparsity and modality-aware kv cache compression for vision-language model inference acceleration. *arXiv preprint arXiv:2410.23317*, 2024.
- Zhongwei Wan, Hui Shen, Xin Wang, Che Liu, Zheda Mai, and Mi Zhang. Meda: Dynamic kv cache allocation for efficient multimodal long-context inference. *arXiv preprint arXiv:2502.17599*, 2025.
- Guangtao Wang, Shubhangi Upasani, Chen Wu, Darshan Gandhi, Jonathan Li, Changran Hu, Bo Li, and Urmish Thakker. Llms know what to drop: Self-attention guided kv cache eviction for efficient long-context inference. *arXiv preprint arXiv:2503.08879*, 2025.
- Ke Wang, Juntong Pan, Weikang Shi, Zimu Lu, Houxing Ren, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. Measuring multimodal mathematical reasoning with math-vision dataset. *Advances in Neural Information Processing Systems*, 37:95095–95169, 2024.
- Zichen Wen, Yifeng Gao, Shaobo Wang, Junyuan Zhang, Qintong Zhang, Weijia Li, Conghui He, and Linfeng Zhang. Stop looking for important tokens in multimodal language models: Duplication

matters more. *arXiv preprint arXiv:2502.11494*, 2025.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Minglai Yang, Ethan Huang, Liang Zhang, Mihai Surdeanu, William Yang Wang, and Liangming Pan. How is llm reasoning distracted by irrelevant context? an analysis using a controlled benchmark. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13340–13358, 2025a.

Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. Visionzip: Longer is better but not necessary in vision language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19792–19802, 2025b.

Yi Yang, Xiaoxuan He, Hongkun Pan, Xiyan Jiang, Yan Deng, Xingtao Yang, Haoyu Lu, Dacheng Yin, Fengyun Rao, Minfeng Zhu, et al. R1-onevision: Advancing generalized multimodal reasoning through cross-modal formalization. *arXiv preprint arXiv:2503.10615*, 2025c.

Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, Yu Su, Wenhui Chen, and Graham Neubig. MMMU-pro: A more robust multi-discipline multimodal understanding benchmark. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15134–15186, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.736.

Yuxuan Yue, Zhihang Yuan, Haojie Duanmu, Sifan Zhou, Jianlong Wu, and Liqiang Nie. Wkvquant: Quantizing weight and key/value cache for large language models gains more. *arXiv preprint arXiv:2402.12065*, 2024.

Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. Lmms-eval: Reality check on the evaluation of large multimodal models, 2024a. URL <https://arxiv.org/abs/2407.12772>.

Qizhe Zhang, Mengzhen Liu, Lichen Li, Ming Lu, Yuan Zhang, Junwen Pan, Qi She, and Shanghang Zhang. Beyond attention or similarity: Maximizing conditional diversity for token pruning in mllms. *arXiv preprint arXiv:2506.10967*, 2025.

Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417*, 2024b.

Yuanhan Zhang, Jinming Wu, Wei Li, Bo Li, Zejun Ma, Ziwei Liu, and Chunyuan Li. Video instruction tuning with synthetic data, 2024c. URL <https://arxiv.org/abs/2410.02713>.

Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. Cam: Cache merging for memory-efficient llms inference. In *Forty-first International Conference on Machine Learning*, 2024d.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.

Meizhi Zhong, Xikai Liu, Chen Zhang, Yikun Lei, Yan Gao, Yao Hu, Kehai Chen, and Min Zhang. Zigzagkv: Dynamic kv cache compression for long-context modeling based on layer uncertainty. In *Proceedings of the 31st International Conference on Computational Linguistics*, 2025.

Appendix of “Make Each Token Count: Towards Improving Long-Context Performance with KV Cache Eviction”

Table of Contents

A	Theoretical Results	22
A.1	Proofs for Attention Dilution	22
A.2	Geometric Retention as Query-Agnostic Future Utility	23
B	Experiments	28
B.1	General Experiment Settings	28
B.2	Baselines	28
B.3	Needle in a Haystack Experiment Settings	28
C	Additional Experimental Results	28
C.1	MMDU	28
C.2	Long Context Benchmarks	29
C.2.1	LongBench-V2	29
C.3	Ablation Study on Lookahead Step (T-t)	29
C.4	Efficiency Scaling	30
C.5	Visualization of DBTrimKV	30
D	Limitations and Future works	32

A. Theoretical Results

A.1. Proofs for Attention Dilution

We now provide proofs for results in Section A.1.

Proof of Proposition 3.1. Let $m = \max_{i \in U_t} z_{t,i}$. Then

$$\sum_{i \in U_t} e^{z_{t,i}} \leq |U_t| e^m, \quad \sum_{j \in C_t \setminus U_t} e^{z_{t,j}} \geq \sum_{d \in D'_t} e^{z_{t,d}} \geq |D'_t| e^{m-\Delta}.$$

Hence

$$\sum_{i \in U_t} \alpha_{t,i} = \frac{\sum_{i \in U_t} e^{z_{t,i}}}{\sum_{j \in C_t} e^{z_{t,j}}} \leq \frac{|U_t| e^m}{|U_t| e^m + |D'_t| e^{m-\Delta}} = \frac{1}{1 + e^{-\Delta} |D'_t| / |U_t|}.$$

Therefore,

$$\delta_t = 1 - \sum_{i \in U_t} \alpha_{t,i} \geq \frac{e^{-\Delta} |D'_t| / |U_t|}{1 + e^{-\Delta} |D'_t| / |U_t|}.$$

If $\Delta = O(1)$, $|U_t| = O(1)$, and $|D'_t| \rightarrow \infty$, then $e^{-\Delta} |D'_t| / |U_t| \rightarrow \infty$, so $\delta_t \rightarrow 1$ and $\sum_{i \in U_t} \alpha_{t,i} = 1 - \delta_t \rightarrow 0$. $\square$

Proof of Proposition 3.2. By definition,

$$\frac{\sum_{i \notin U_t} e^{z_{t,i}}}{\sum_{i \in U_t} e^{z_{t,i}}} = \frac{\delta_t}{1 - \delta_t}.$$

Using the definitions of ρ_U and ρ_D ,

$$\delta_t^r = \frac{\rho_D \sum_{i \notin U_t} e^{z_{t,i}}}{\rho_U \sum_{i \in U_t} e^{z_{t,i}} + \rho_D \sum_{i \notin U_t} e^{z_{t,i}}}.$$

Dividing numerator and denominator by $\rho_U \sum_{i \in U_t} e^{z_{t,i}}$ gives

$$\delta_t^r = \frac{(\rho_D / \rho_U) \delta_t}{(1 - \delta_t) + (\rho_D / \rho_U) \delta_t}.$$

If $\rho_D \leq \rho_U$, then $\rho_D / \rho_U \leq 1$, and since $a \mapsto a \delta_t / ((1 - \delta_t) + a \delta_t)$ is nondecreasing for $a \geq 0$,

$$\delta_t^r \leq \frac{\delta_t}{(1 - \delta_t) + \delta_t} = \delta_t.$$

Finally, if $\rho_D / \rho_U \rightarrow 0$ and $1 - \delta_t > 0$, then the displayed formula gives

$$\delta_t^r \rightarrow 0.$$

$\square$

A.2. Geometric Retention as Query-Agnostic Future Utility

We now justify the geometric retention form $r_{t,i} = \beta_i^{t-i}$ used by retention-gated attention. The central idea is to view the usefulness of an old cached token as a persistence event. A token should be retained if it is likely to remain useful, or close to useful, for future query states. Under stable future query dynamics, this persistence probability decays geometrically whenever the token has a uniformly positive chance of leaving its useful region over a bounded time window.

We adopt a one-shot compression viewpoint. At decoding step t , the existing cache

$$C_t = \{1, \dots, t\}$$

is compressed to a smaller set of old tokens. Tokens generated after time t are not part of this compression decision: they may be retained automatically or handled by later compression rounds. Thus, when deciding whether an old token $i \leq t$ should survive compression at time t , we compare it only against the other old cached tokens in C_t .

For each old cached token $i \leq t$, define its old-cache future utility by

$$\bar{G}_i(t) := \sum_{s=t+1}^T w_{t,s} \Pr(i \in U_s^{(t)} \mid \mathcal{F}_t), \quad (10)$$

where $w_{t,s} \geq 0$ are horizon weights and $U_s^{(t)} \subseteq C_t$ denotes the set of old cached tokens that remain useful at future step s . This quantity measures the expected future benefit of retaining token i after compressing the old cache at time t .

Assumption A.1 (Stable low-rank future query dynamics). *For a fixed attention head and conditioning on $\mathcal{F}_t$, suppose the future query-side state evolves in a low-dimensional subspace:*

$$z_{s,i} = \mathbf{x}_s^\top (\mathbf{W}_Q^\top \mathbf{W}_K) \mathbf{x}_i \approx \mathbf{r}_s^\top \mathbf{c}_i + \varepsilon_{s,i}, \quad \mathbf{r}_s \in \mathbb{R}^m, \quad \mathbf{c}_i \in \mathbb{R}^m,$$

where $m \ll d$. The future process $(\mathbf{r}_s)_{s>t}$ is a stable, time-homogeneous Markov chain on $\mathbb{R}^m$ with transition kernel P and invariant distribution π . Stability means that the chain is mean-reverting and ergodic, so that its distribution converges to π from typical initial states.

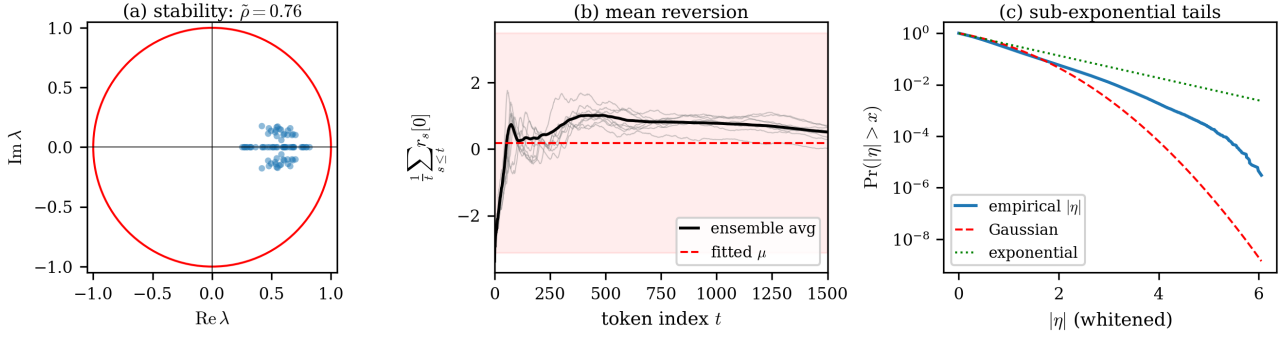


Figure 8: Empirical evidence for Assumption A.1. We extract the per-token query state $\mathbf{q}_t$ at layer 20, head 0 of Qwen3-8B for 128 rollouts each on 10 AIME-24 prompts, project onto the top-8 PCA directions of the pooled training queries to obtain $\mathbf{r}_t \in \mathbb{R}^8$, and fit a per-context VAR(1) model $\mathbf{r}_{t+1} = A\mathbf{r}_t + \mathbf{b} + \boldsymbol{\eta}t$ on 75% of rollouts. **(a) Stability.** Eigenvalues of A across all 10 contexts (each context contributes 8 eigenvalues) lie strictly inside the unit circle; the median spectral radius is $\tilde{\rho} = 0.76$ and every fitted context is stable, so the chain is mean-reverting. **(b) Mean reversion / ergodicity.** For one context, the per-rollout running mean $\frac{1}{t} \sum_{s \leq t} r_s[0]$ (grey) and its ensemble average (black) converge to the fitted μ (red dashed) inside a $\pm\sigma$ stationary band; the time average matches the ensemble average, consistent with ergodicity. **(c) Sub-exponential innovations.** Pooled log-survival of the whitened innovation magnitude $|\eta|$ (blue) sits strictly between the Gaussian (red dashed) and exponential (green dotted) reference curves, satisfying the sub-exponential tail condition that drives the persistence bound.

Under Assumption A.1, future attention scores against old cached tokens are determined, up to approximation error, by the future query state $\mathbf{r}_s$ and the fixed compatibility vectors $\{\mathbf{c}_j : j \leq t\}$. Hence top- K membership among old cached tokens can be described as a region in the future query-state space.

For a future query state $\mathbf{r}$, define the hard old-cache top- K threshold

$$\Gamma_{K,t}(\mathbf{r}) := K\text{-th largest value in } \{\mathbf{r}^\top \mathbf{c}_j : j \in C_t\}.$$

We say that token $i \leq t$ is immediately useful at query state $\mathbf{r}$ if it lies in the old-cache top- K , namely if

$$\mathbf{r}^\top \mathbf{c}_i \geq \Gamma_{K,t}(\mathbf{r}).$$

Equivalently, the hard top- K useful region for token i is

$$\mathcal{T}_{i,t}^{(K)} := \left\{ \mathbf{r} \in \mathbb{R}^m : \mathbf{r}^\top \mathbf{c}_i \geq \Gamma_{K,t}(\mathbf{r}) \right\}.$$

If $\mathbf{r} \in \mathcal{T}_{i,t}^{(K)}$, then token i is among the top- K old cached tokens for query state $\mathbf{r}$.

However, immediate top- K membership is too strict as a retention criterion. A token that is not currently in the top- K may still be worth retaining if it remains close to the top- K boundary or if plausible future query states can make it top- K again. Retention should therefore capture not only instantaneous usefulness, but also future usefulness under small changes in the query state.

To formalize this idea, we introduce a token-dependent relaxed threshold

$$\tilde{\Gamma}_{i,K,t}(\mathbf{r}) \leq \Gamma_{K,t}(\mathbf{r}).$$

Equivalently, we write

$$\tilde{\Gamma}_{i,K,t}(\mathbf{r}) = \Gamma_{K,t}(\mathbf{r}) - \Delta_{i,t}(\mathbf{r}), \quad \Delta_{i,t}(\mathbf{r}) \geq 0,$$

where $\Delta_{i,t}$ is a token-dependent slack. The hard top- K rule is recovered when $\Delta_{i,t} \equiv 0$.

This slack has the following interpretation. Token i is useful in the hard sense when it is in the old-cache top- K . For retention, we lower the hard threshold by the minimal amount needed to regard token i as still useful along the future query trajectory we wish to preserve. Thus the relaxed threshold can be viewed as the least permissive relaxation of the top- K boundary under which token i remains retention-worthy.

Define the relaxed survival region

$$\mathcal{S}_{i,t}^{(K)} := \left\{ \mathbf{r} \in \mathbb{R}^m : \mathbf{r}^\top \mathbf{c}_i \geq \tilde{\Gamma}_{i,K,t}(\mathbf{r}) \right\}.$$

Since

$$\tilde{\Gamma}_{i,K,t}(\mathbf{r}) \leq \Gamma_{K,t}(\mathbf{r}),$$

we have

$$\mathcal{T}_{i,t}^{(K)} \subseteq \mathcal{S}_{i,t}^{(K)}.$$

Thus, every query state in which token i is hard top- K is also in its relaxed survival region, while $\mathcal{S}_{i,t}^{(K)}$ may additionally include states where token i is not top- K but remains sufficiently compatible with the query to justify retaining it. This definition separates immediate usefulness from retention-worthiness: the hard top- K region identifies tokens that are currently among the strongest old-cache competitors, while the relaxed survival region identifies tokens that remain close enough to this condition to be worth keeping. We thus have, for a fixed compression time t and a future step $s > t$, define

$$U_s^{(t)} := \left\{ i \leq t : \mathbf{r}_u \in \mathcal{S}_{i,t}^{(K)} \text{ for all } u = t+1, \dots, s \right\}.$$

This persistence formulation is natural for monotone KV eviction. Once an old token is removed during compression, it cannot re-enter the cache later. Hence a retained token should not merely be useful for one future query; it should remain within a region where it is likely to be useful over the intervening future query trajectory.

In autoregressive generation, token relevance is typically local and query-dependent. Many cached tokens are useful only for a limited phase of the generation: they may be highly compatible with the current query, but as the query state drifts toward later semantic or syntactic contexts, they naturally leave the region in which they are useful (see Figure 9). This motivates the following block-exit condition on the relaxed survival region.

Assumption A.2 (Top- K block exit). *Fix a compression time t . For each old cached token $i \leq t$, there exist an integer $b_i \geq 1$ and a constant $\epsilon_i > 0$ such that*

$$\sup_{\mathbf{r} \in \mathcal{S}_{i,t}^{(K)}} \Pr(\mathbf{r}_1, \dots, \mathbf{r}_{b_i} \in \mathcal{S}_{i,t}^{(K)}) \leq 1 - \epsilon_i.$$

Here $\Pr_{\mathbf{r}}$ denotes the law of the Markov chain initialized at $\mathbf{r}_0 = \mathbf{r}$. Equivalently, whenever token i is currently inside its relaxed survival region, there is probability at least ϵ_i that it exits this region within the next b_i future query steps.

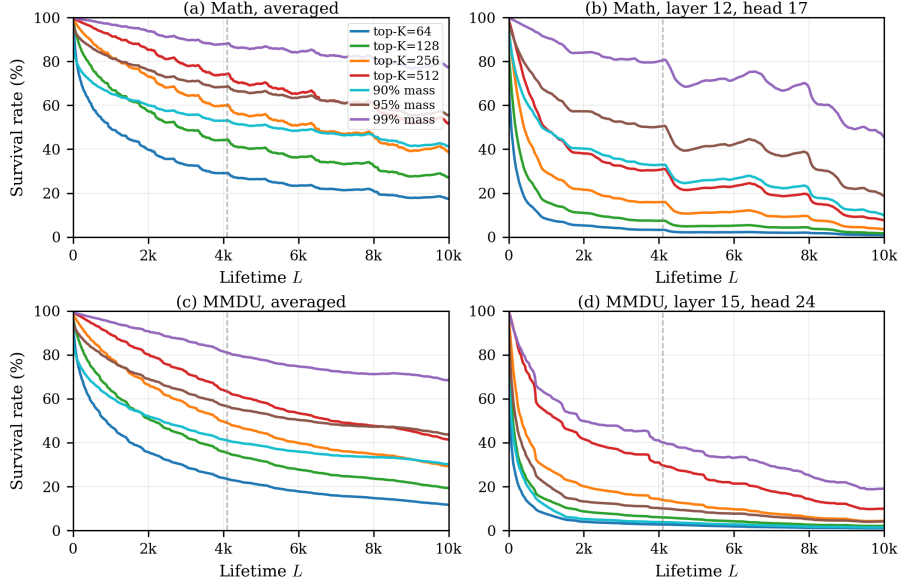


Figure 9: Token attention-survival under full-cache inference. **Top row:** Qwen3-4B on 100 OpenR1-Math prefills up to $T = 16,384$ tokens. **Bottom row:** Qwen3-VL-4B on 98 MMDU multimodal dialogues (text + interleaved images), prefilled to the same length. **Left column:** survival rate averaged across all layers and query heads. **Right column:** survival rate for a single representative head (Math: layer 12, head 17; MMDU: layer 15, head 24). For each query position p we identify the head’s selected set of past tokens using either the top- K logits ($K \in 64, 128, 256, 512$) or the smallest set whose softmax mass reaches a threshold $\tau \in 0.90, 0.95, 0.99$; a token of birth time i is then alive at horizon L if it appears in the selected set of some query at position $p \geq i + L$. The dashed vertical line marks $L = 4096$. In every panel, every fixed top- K curve drops sharply with L , while the more lenient mass criteria stay substantially higher: the gap between top- K and τ -mass curves is the attention-dilution mass — tokens that still carry meaningful aggregate weight but are pushed out of the head’s hard top- K budget. Heterogeneity at the head level is large: a representative head on either dataset loses the majority of tokens by $L = 4096$ even under the most lenient criterion (99% mass), in contrast to the across-head average. The same qualitative shape appears in the multimodal setting (rows c, d), confirming that attention dilution is not specific to text-only inputs.

The block length b_i is allowed to be token dependent. For tokens whose relevance is tied to the current local query context, b_i may be small. For globally useful tokens, such as sink tokens, delimiter tokens, or other persistent structural tokens, b_i may be much larger, reflecting the slower decay of their usefulness.

Theorem A.3 (Exponential relaxed top- K persistence). *Suppose Assumptions A.1 and A.2 hold, and define $U_s^{(t)}$ through relaxed survival regions as above. Then, for each old cached token $i \leq t$, there exist constants $A_i > 0$ and $\theta_i > 0$ such that, for all $n \geq 1$,*

$$\Pr(i \in U_{t+n}^{(t)} \mid \mathcal{F}_t) \leq A_i e^{-\theta_i n}.$$

In particular, one may take

$$\theta_i = -\frac{1}{b_i} \log(1 - \epsilon_i), \quad A_i = (1 - \epsilon_i)^{-1}.$$

Equivalently, defining

$$\beta_i := e^{-\theta_i} = (1 - \epsilon_i)^{1/b_i} \in (0, 1),$$

the old-cache future utility in Eq. (10) satisfies

$$\bar{G}_i(t) \leq A_i \sum_{s=t+1}^T w_{t,s} \beta_i^{s-t}.$$

Proof. Let

$$S_n := \left\{ \mathbf{r}_{t+1}, \dots, \mathbf{r}_{t+n} \in \mathcal{S}_{i,t}^{(K)} \right\}$$

be the event that token i 's future query trajectory remains inside its relaxed survival region for n consecutive future steps. By the definition of $U_s^{(t)}$,

$$\Pr(i \in U_{t+n}^{(t)} \mid \mathcal{F}_t) \leq \Pr(S_n \mid \mathcal{F}_t).$$

Write $n = qb_i + r$, where $q = \lfloor n/b_i \rfloor$ and $0 \leq r < b_i$. Applying the Markov property at the beginning of each block and using Assumption A.2 gives

$$\Pr(S_n \mid \mathcal{F}_t) \leq \Pr(S_{qb_i} \mid \mathcal{F}_t) \leq (1 - \epsilon_i)^q.$$

Since $q \geq n/b_i - 1$,

$$(1 - \epsilon_i)^q \leq (1 - \epsilon_i)^{-1} \exp\left(-\frac{-\log(1 - \epsilon_i)}{b_i} n\right).$$

Therefore,

$$\Pr(S_n \mid \mathcal{F}_t) \leq A_i e^{-\theta_i n},$$

with

$$A_i = (1 - \epsilon_i)^{-1}, \quad \theta_i = -\frac{1}{b_i} \log(1 - \epsilon_i).$$

The bound on $\bar{G}_i(t)$ follows by substituting this persistence bound into Eq. (10). $\square$

Theorem A.3 gives a probabilistic interpretation of geometric retention. The survival parameter

$$\beta_i = (1 - \epsilon_i)^{1/b_i}$$

summarizes how persistent token i 's future usefulness is expected to be. Tokens that quickly exit their relaxed top- K survival regions have smaller β_i , while tokens that remain compatible with many future query states have β_i closer to one.

In practice, the block-exit parameters (b_i, ϵ_i) are not estimated explicitly. Instead, the retention gate learns a direct prediction of β_i from the token representation, or equivalently from features that determine its compatibility vector $\mathbf{c}_i$. Thus, β_i can be viewed as a learned summary of the geometry of the relaxed top- K survival region $\mathcal{S}_{i,t}^{(K)}$ under the future query dynamics.

B. Experiments

B.1. General Experiment Settings

We train with a maximum sequence length of 32,768, using a learning rate of 2×10^{-4} , weight decay 10^{-6} , and cosine scheduling. The per-GPU batch size is 1 with gradient accumulation set to 4; all other hyperparameters follow the default HuggingFace Trainer settings. For the training datasets, we randomly sample 40% of M4Instruct [Li et al., 2024a], 30% of Academic Video [Zhang et al., 2024c], 30% of R1-OneVision [Yang et al., 2025c], and 20% of OpenR1-Math-220k [Hugging Face]. The training is performed on 4 H100 GPUs.

B.2. Baselines

- SnapKV [Li et al., 2024c] is an attention-score-based eviction heuristic, and R-KV extends this approach to reasoning models by identifying redundant tokens via key-vector similarity.
- AdaKV [Feng et al., 2024] builds on SnapKV with dynamic, head-wise budget allocation driven by attention statistics.
- R-KV [Cai et al., 2025] is a training-free compression method that optimizes the KV cache of reasoning models by jointly scoring tokens based on attention importance and semantic redundancy.
- TrimKV is our extension of the token retention strategy from [Bui et al., 2025] to vision-language models.

B.3. Needle in a Haystack Experiment Settings

To demonstrate how DBTrimKV mitigates attention dilution by evicting distractors, we follow the Needle-in-a-Haystack (NIAH) experimental setup [Hsieh et al., 2024]. Because most recent LLMs have achieved near-perfect accuracy on single-NIAH tasks, we increase the difficulty by combining MK-NIAH, MV-NIAH, and MQ-NIAH into a single setting. We generate a dataset of 5,117 training samples with varying context sizes (from 4k to 200k) and randomized numbers of keys, values, and queries. For testing, we hold out 200 samples where the numbers of keys, values, and queries are fixed to 8, 4, and 4, respectively. Using Qwen3-4B-Instruct-2507 (which supports a 256k context window) as our base model, we evaluate two trainable KV eviction methods: TrimKV and DBTrimKV. Similar to the main experiments, we freeze the base model and train only the retention head.

C. Additional Experimental Results

C.1. MMDU

Table 6 reports the complete set of LLM-as-a-judge metrics in MMDU [Liu et al., 2024e], serving as a detailed extension of Table 3.

The extended metrics reveal several critical trends regarding model performance under extreme sequence compression. First, DBTrimKV consistently outperforms not only the competing eviction methods but also the Vanilla full-cache baseline across all tested KV budgets (512, 256, and 128). Notably, even under the extreme constraint of a 128-token budget, DBTrimKV maintains an overall performance of 104.10% relative to the Vanilla baseline, whereas existing methods like SnapKV, R-KV, and AdaKV suffer severe degradation, dropping to 60–64% relative performance.

Furthermore, the metric breakdown highlights the specific failure modes of traditional eviction strategies. As the budget decreases, baseline methods exhibit sharp declines in multimodal-specific metrics such as “Visual Perception” and “Image Relative,” indicating a failure to retain visually grounded context. In

contrast, both TrimKV and DBTrimKV preserve these critical tokens, allowing DBTrimKV to actually exceed Vanilla performance in visual perception (3.54 vs. 3.40) and logical coherence (4.11 vs. 3.90) at the lowest 128 budget.

Budget	Method	Answer Accuracy	Creativity	Richness	Visual Perception	Logical Coherence	Image Relative	Overall	% vs Vanilla
	Vanilla	3.37	3.68	4.21	3.40	3.90	3.40	3.57	100
512	SnapKV	3.07	<u>4.13</u>	<u>4.74</u>	2.78	3.88	3.05	3.46	96.70
	R-KV	2.76	3.32	3.77	2.58	3.43	2.78	3.00	84.00
	AdaKV	2.89	<u>3.79</u>	<u>4.32</u>	2.68	3.68	2.93	3.24	90.79
	TrimKV ^o	<u>3.46</u>	<u>3.88</u>	<u>4.25</u>	<u>3.51</u>	<u>4.00</u>	<u>3.64</u>	<u>3.73</u>	<u>104.51</u>
	DBTrimKV ^o	3.79	4.25	4.69	3.88	4.38	3.91	4.09	114.46
256	SnapKV	2.54	3.44	4.20	2.23	3.32	2.40	2.88	80.51
	R-KV	2.42	2.99	3.58	2.22	3.10	2.36	2.66	74.45
	AdaKV	2.56	3.41	4.16	2.25	3.31	2.45	2.88	80.68
	TrimKV ^o	<u>3.16</u>	<u>3.98</u>	<u>4.25</u>	<u>3.35</u>	<u>3.97</u>	<u>3.63</u>	<u>3.56</u>	<u>99.65</u>
	DBTrimKV ^o	3.64	4.09	4.54	3.74	4.18	3.73	3.87	108.33
128	SnapKV	2.01	2.59	3.31	1.72	2.85	1.85	2.29	64.00
	R-KV	2.02	2.21	2.63	1.93	2.71	2.06	2.17	60.77
	AdaKV	2.03	2.54	3.21	1.78	2.83	1.92	2.25	62.98
	TrimKV ^o	<u>3.05</u>	<u>3.85</u>	<u>4.11</u>	<u>3.03</u>	<u>3.88</u>	<u>3.41</u>	<u>3.35</u>	<u>93.61</u>
	DBTrimKV ^o	3.43	3.93	4.31	3.54	4.11	3.57	3.72	104.10

Table 6: Comparison of different methods across various metrics under varying token budgets. The best eviction methods are shown in **bold**, and the second-best are underlined. Performances that exceed vanilla inference are highlighted in **red**. The superscript ^o denotes a trainable KV eviction method.

C.2. Long Context Benchmarks

C.2.1. LongBench-V2

In this section, we compare our method against Full KV Cache, TrimKV, and Locret [Huang et al., 2025b] on LongBenchv2 benchmark [Bai et al., 2024]. We follow the chunk-prefill settings from [Huang et al., 2025b]. Base model is microsoft/Phi-3-mini-128k-instruct, locret model is from hyx21/Locret-phi-3-mini-128K. LongBench-v2 evaluates deep reasoning across realistic long-context scenarios, such as multi-document QA and code repository comprehension. Locret is a lightweight, training-based eviction method that employs learnable retaining heads to predict the causal importance of tokens. As shown in Table 7, DBTrimKV achieves 9.20% average accuracy improvement over the Full KV baseline, whereas both Locret and TrimKV experience performance degradation.

Method	Acc. Short	Acc. Easy	Acc. Medium	Acc. Hard	Avg. Acc	Avg. Δ (%)
Full KV	<u>33.71</u>	<u>34.44</u>	18.60	25.86	<u>28.79</u>	<u>0.00</u>
LocRet	30.34	30.00	24.42	27.59	28.41	-1.32
TRIM-KV	29.21	31.11	<u>19.77</u>	23.56	26.13	-9.24
DBTrimKV	37.08	36.67	<u>19.77</u>	28.74	31.44	+9.20

Table 7: Performance on long-context tasks of KV eviction methods with Phi3-mini-128K on the LongBench-V2 benchmark, including average relative change (Avg. Δ) compared to Full KV. KV Budget is set to 1024.

C.3. Ablation Study on Lookahead Step (T-t)

During inference, we use a default lookahead horizon $T - t = 2$ (as described in Section B.1) for every eviction time t . We now ablate the effect of this lookahead parameter on the MathVision_{mini} benchmark across various KV budgets. As shown in Table 8, a lookahead step of $T - t = 2$ generally yields the

Lookahead/Budget	64	128	256	512	1024
DBTrimKV - LHS $T - t = 1$	34.54	46.43	49.34	50.33	50.66
DBTrimKV - LHS $T - t = 2$	33.88	47.04	51.64	51.97	52.63
DBTrimKV - LHS $T - t = 5$	38.49	47.73	47.70	50.00	46.05

Table 8: Lookahead-step $T - t$ ablation on MathVision_{mini}. Here, t is the compression time.

highest accuracy for moderate to high KV budgets (256, 512, and 1024). Reducing the lookahead to a myopic one-step horizon ($T - t = 1$) consistently degrades performance across these budgets. Conversely, extending the lookahead horizon to $T - t = 5$ harms performance at higher budgets, such as dropping from 52.63% to 46.05% at a budget of 1024. However, under extremely constrained memory budgets (64 and 128), the longer lookahead horizon of $T - t = 5$ becomes beneficial, outperforming both $T - t = 1$ and $T - t = 2$. This indicates that a moderate lookahead optimally balances recent context with long-term retention under typical memory conditions, whereas a longer lookahead helps prioritize the most globally persistent tokens when cache capacity is severely limited.

C.4. Efficiency Scaling

To evaluate inference efficiency, we benchmark in a synthetic setting using dummy input sequences of varying context lengths, populated with filler tokens and an all-ones attention mask. We compare three methods: Vanilla (Full Cache), our proposed DBTrimKV, and TrimKV. Efficiency is evaluated by measuring total decoding time (seconds) and throughput (generated tokens per second). Timing is recorded during the generation phase and averaged over multiple steps following an initial warmup period. Results are ported in Figure 10, 11, 12.

Across all scaling dimensions, our method, DBTrimKV, demonstrates significant efficiency improvements over the Vanilla baseline. As context length grows, the Vanilla model suffers from linearly increasing decoding times and decreasing throughput, whereas DBTrimKV maintains a consistent, flat efficiency profile. When scaling generation length, Vanilla exhibits a severe, quadratic increase in decoding time, whereas DBTrimKV scales much more efficiently, maintaining significantly higher throughput and lower decoding times even at extended lengths. Finally, across varying KV budgets, DBTrimKV consistently retains its efficiency advantage over the static Full Cache baseline, with throughput and decoding time remaining largely stable regardless of the specific budget allocation.

C.5. Visualization of DBTrimKV

In this section, we visualize the visual tokens retained during inference to illustrate the model’s selection behavior. As shown in Figures 13 and 14, DBTrimKV initially preserves a high density of visual tokens during the early stages of generation to establish structural and semantic understanding (e.g., Figure 13a).

As decoding progresses, DBTrimKV aggressively evicts background distractors, narrowing its focus strictly to the most informative image regions necessary for the ongoing reasoning task (e.g., Figure 13b). Notably, the retained visual tokens align directly with the model’s textual output. In Figure 13, retention is heavily concentrated on the scythe, the clock, and the specific "RATHER TIME" text banner, which the model explicitly relies upon to deduce the final answer. Similarly, in Figure 14, retention isolates the necessary coordinate data points. This dynamic focusing empirically validates that our learned retention mechanism successfully mitigates attention dilution by preserving only tokens with long-term utility while discarding irrelevant visual context. This trend also aligns with the layer-wise attention patterns visualized in Figure 6, where a massive initial KV cache is gradually pruned as generation

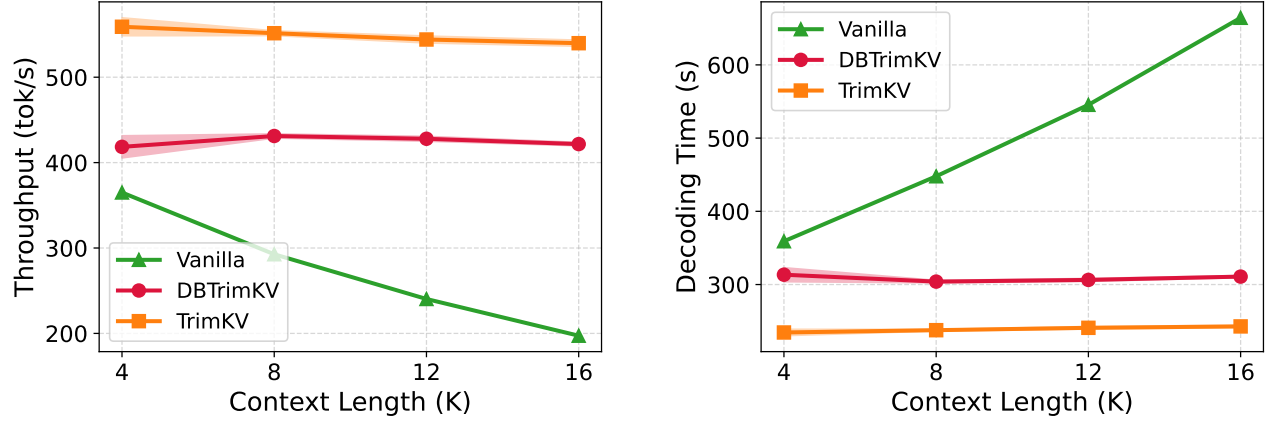


Figure 10: Efficiency scaling with context length. The figure reports throughput in tokens per second (left) and total decoding time in seconds (right) as the input context length increases. Evaluated methods include Vanilla (Full Cache), DBTrimKV, and TrimKV. We fix batch size at 16, number of generated tokens at 8192 and KV budget at 256. Results are averaged over 5 random seeds.

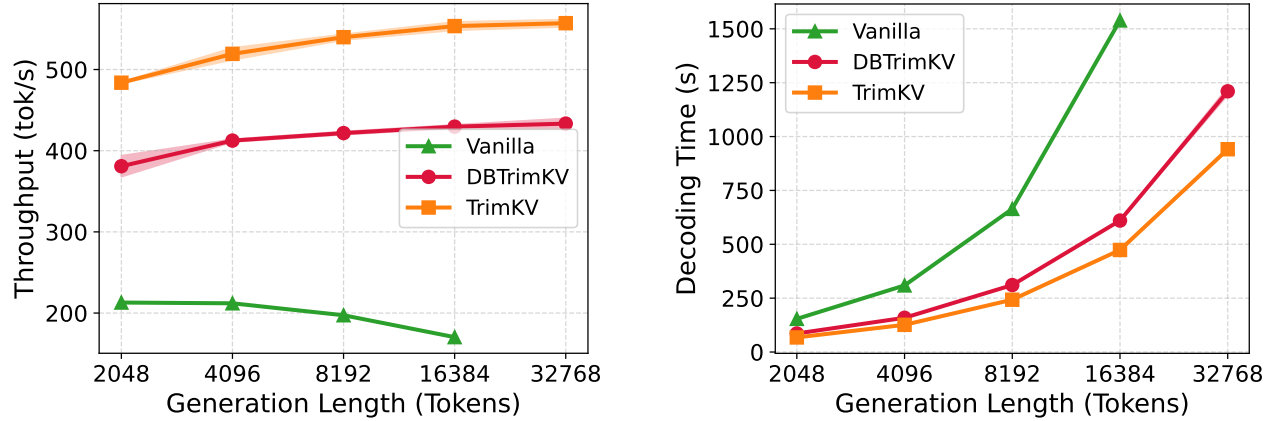


Figure 11: Efficiency scaling with generation length. The figure reports throughput in tokens per second (left) and total decoding time in seconds (right) as the number of generated tokens increases. Evaluated methods include Vanilla (Full Cache), DBTrimKV, and TrimKV. We fix batch size at 16, context length at 16384 and KV budget at 256. We do not report vanilla performance for generation length at 32k due to OOM error. Results are averaged over 5 random seeds.

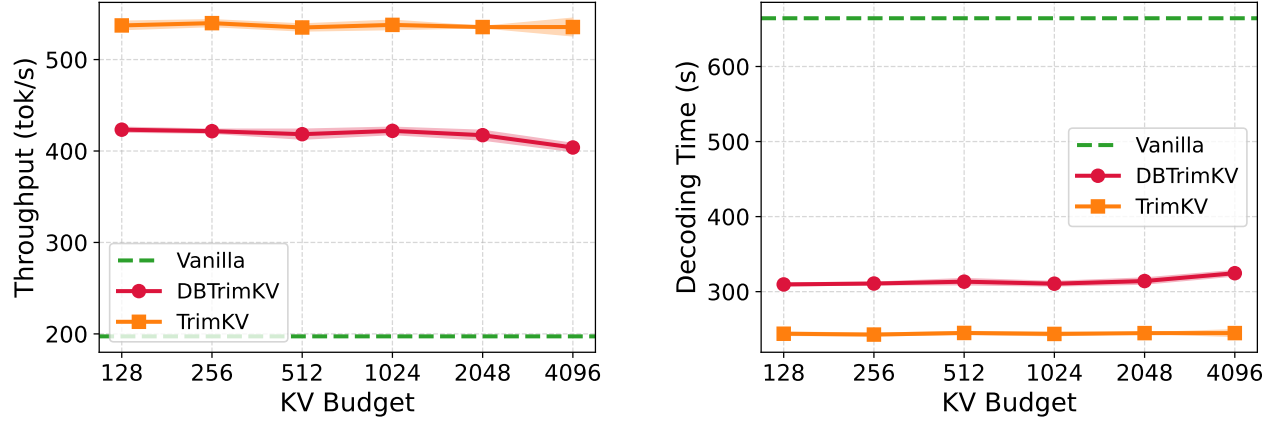


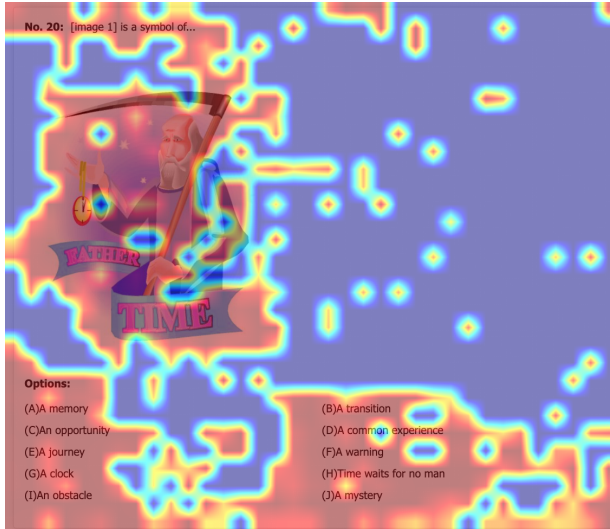
Figure 12: Efficiency scaling with KV budget. The figure reports throughput in tokens per second (left) and total decoding time in seconds (right) across different allocated KV memory budgets. Evaluated methods include Vanilla (Full Cache), DBTrimKV, and TrimKV. We fix batch size at 16, number of generated tokens at 8192 and number of generated tokens at 8192. Results are averaged over 5 random seeds.

continues.

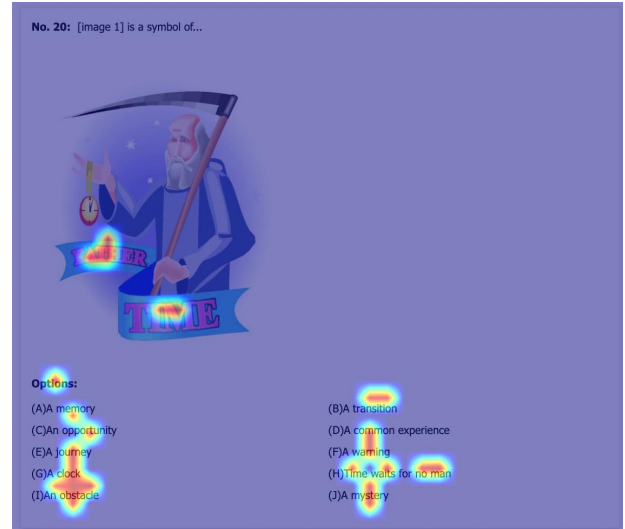
D. Limitations and Future works

While our proposed DBTrimKV demonstrates strong performance, there are several limitations and avenues for future exploration:

- In our current methodology, we only train the lightweight retention gates and freeze the base language model weights. While this approach is computationally efficient and preserves the fundamental capabilities of the base model, the impact of jointly fine-tuning the base LLM alongside the retention network remains unknown. Future work should investigate whether end-to-end joint training allows the model to inherently structure its internal representations to be more amenable to KV cache eviction.
- Our empirical evaluations were conducted on models up to the 8B parameter scale. We did not study the efficacy of globally calibrated retention gates on significantly larger, frontier-scale models. Establishing scaling laws for token retention and evaluating whether the mitigation of attention dilution behaves similarly at the 70B+ parameter scale is a critical next step.
- We observe that DBTrimKV can surpass full-cache inference performance at specific low-budget regimes by effectively suppressing distractor tokens. This indicates that token eviction can be utilized not just for compression, but as a mechanism to actively improve reasoning. Future work could explore using reinforcement learning as a post-training step, treating token retention as an action space to actively explore and optimize eviction policies for downstream performance.



(a) Generation step 15

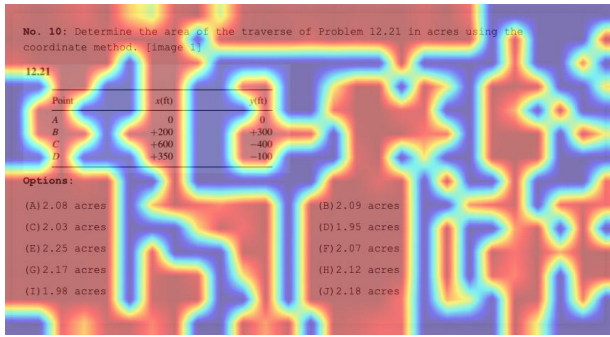


(b) Last generation step

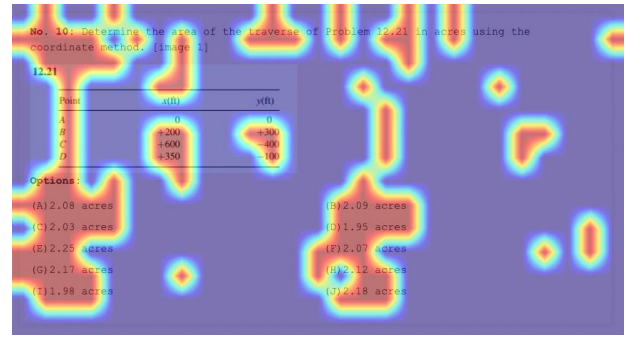
let's look at the image. The character is holding a scythe (which is usually associated with Death) but also a clock, and the banner says "RATHER TIME". Wait, the options include (H) Time waits for no man. That's a common saying. Let me check the options again. The image has "RATHER TIME" which might be a play on "Time waits for no man". So the symbol here is representing that phrase. So the answer should be H.

(c) Model reasoning

Figure 13: Visualization of retained visual tokens for a sample in the MMMU-Vision-Pro dataset (Layer 13, Head 2). While the initial step (a) retains broad context, the final step (b) aggressively prunes background noise to isolate the scythe, the clock, and the “RATHER TIME” banner. This precise visual retention directly grounds the model’s textual deduction (c) that the symbol represents the phrase “Time waits for no man” (Option H).



(a) Generation step 15



(b) Last generation step

let's solve this problem using the coordinate method... $\text{Area} = (1/2) \left| \sum_{i=1}^n (x_i y_{i+1} - x_{i+1} y_i) \right| \dots$ The problem gives points A, B, C, D. Let's confirm the order: A(0,0), B(200,300), C(600,-400), D(350,-100)... $\text{Area} = (1/2) \left| -180,000 \right| = 90,000$ square feet... Now, we need to convert square feet to acres. We know that 1 acre = 43,560 square feet. So, $\text{Area in acres} = 90,000 / 43,560 \approx 2.066$... So 2.066 is approximately 2.07, which is option F.

(c) Model reasoning

Figure 14: Visualization of retained visual tokens for a sample in the MMMU-Vision-Pro dataset (Layer 3, Head 3). The final generation step (b) demonstrates DBTrimKV isolating the specific textual coordinate data points (e.g., A(0,0), B(200,300)) required by the model to formulate the area computation in its reasoning output (c). The correct answer is F.